# NETWORK PROBE ATTACK PREDICTION USING MACHINE LEARNING

A PROJECT REPORT SUBMITTED TO

**SRM INSTITUTE OF SCIENCE & TECHNOLOGY**

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

**MASTER OF COMPUTER APPLICATIONS**

By

SAIDEEPA NV

REG NO. DA2232306010006

UNDER THE GUIDANCE OF

**Dr. A THIRUMURTHI RAJA M.C.A., M.E., PH.D.,**

**ASSISTANT PROFESSOR**,

Department of Computer Applications,

Directorate of Distance Education,

SRMIST, Chennai

**DEPARTMENT OF COMPUTER APPLICATIONS, FACULTY OF SCIENCE AND HUMANITIESSRM INSTITUTE OF SCIENCE & TECHNOLOGY,**

Kattankulathur – 603 203

Chennai, Tamilnadu

**June - 2024**

# BONAFIDE CERTIFICATE

This is to certify that the project report titled "**NETWORK PROBE ATTACK PREDICTION USING MACHINE LEARNING**" is a bonafide work carried out by **SAIDEEPA NV** (REGNO REG NO. DA2232306010006) under my supervision for the award of the Degree of Master of Computer Applications. To my knowledge the work reported herein is the original work done by these students.

| **PROJECT GUIDE** | **PROGRAMME CO-ORDINATOR** |
|---|---|
| **Dr. A.Thirumurthi Raja,** | **Dr. A.Thirumurthi Raja,** |
| ASSISTANT PROFESSOR, | ASSISTANT PROFESSOR, |
| DDE – SRMIST | DDE - SRMIST |
| INTERNAL EXAMINER | EXTERNAL EXAMINER |

# ACKNOWLEDGEMENT

With profound gratitude to the ALMIGHTY, I take this chance to thank the people who helped me to complete this project.

I take this as a right opportunity to say THANKS to my parents who are there to stand with me always with the words “YOU CAN”.

I am thankful to **Dr. T.R. Paarivendhar, Chancellor**, SRM Institute of Science & Technology who gave us the platform to establish me to reach greater heights.

I express my deep sense of gratitude to the Director **Dr. R. Rajagopal,** and overall course Coordinator **Dr. G. Venugopalan** for their wholehearted support and encouragement.

It is our delight to thank my project guide

**Dr. A THIRUMURTHI RAJA M.C.A., M.E., PH.D.,**

Assistant Professor, Department of Computer Applications for his help, support, encouragement, suggestions, and guidance throughout the development phases of the project.

I convey our gratitude to all the faculty members of the department who extended their support through valuable comments and suggestions during the reviews.

A great note of gratitude to friends and people who are known and unknown to me who helped carrying out this project work a successful one.

**SAIDEEPA NV**

# TABLE OF CONTENTS


| TOPICS | PG NO |
|---|---|
| **ABSTRACT** | 7 |
| **CHAPTER 1 - INTRODUCTION**<br>1.1 Introduction of Project – Sensor Network<br>1.2 Sensor Node<br>1.3 WSN STRUCTURE<br>1.4 WSNS with the cluster structure<br>1.5 WSN Gate<br>1.6 Network Topology<br>1.7 General Design Issues<br>1.8 Key Features of Sensor Nodes<br>1.9 Characteristics of the WSN which Utilizes such Sensor Nodes computing Power<br>1.10 Introduction of the Domain<br>1.11 Objective of the Problem<br>1.12 Scope of the project | 8 - 26 |
| **CHAPTER 2 - LITERATURE REVIEW – 2.1 – 2.10** | 27 - 33 |
| **CHAPTER 3 - PROBLEM STATEMENT AND METHODOLOGY**<br>3.1 PROBLEM DEFINITION<br>3.2 METHODOLOGY<br>3.2.1 Existing System<br>3.2.2 Proposed System – Convolution Neural Network<br>3.3 System Architecture<br>3.4 UML Diagram<br>3.4.1 Use case Diagram<br>3.4.2 Class Diagram<br>3.4.3 Component Diagram<br>3.4.4 Deployment Diagram | 34 - 41 |

| | |
|---|---|
| **CHAPTER 4 - SYSTEM IMPLEMENTATION**<br>4.1 MODULE DESCRIPTION<br>4.2 ALGORITHM OF PROPOSED WORK<br>4.3 FLOWCHART<br>4.4 COMPARATIVE STUDY OF EXISTING AND PROPOSED SYSTEM | 42 - 45 |
| **CHAPTER 5 - SYSTEM STUDY**<br>5.1 FEASIBILTY STUDY<br>5.2 ECONOMICAL FEASIBILITY<br>5.3 TECHNICAL FEASIBILITY<br>5.4 SOCIAL FEASIBILITY<br>5.5 OPERATIONAL FEASIBILITY | 45 - 47 |
| **CHAPTER 6 - SYSTEM TESTING - TYPES OF TESTING**<br>6.1 Unit testing<br>6.2 Integration testing<br>6.3 Functional testing<br>6.4 System Testing<br>6.5 White Box Testing<br>6.6 Black Box Testing<br>6.7 Unit Testing<br>6.8 Integration Testing<br>6.9 Acceptance Testing | 48 - 51 |

| | |
|---|---|
| **CHAPTER 7 - SOFTWARE DESCRIPTION PYTHON SOFTWARE**<br>7.1 Introduction to Python<br>7.2 Python Features<br>7.3 Python Environment Setup<br>7.4 Getting Python<br>7.5 Installing Python<br>7.6 Macintosh Installation<br>7.7 Setting up PATH<br>7.8 Running Python<br>7.9 PANDA<br>7.10 Key Features of PANDA | 52 - 66 |
| **CHAPTER 8 - CONCLUSION AND FUTURE SCOPE**<br>8.1 Conclusion<br>8.2 Future Scope | 67 |
| **REFERENCES** | 68 - 69 |
| **APPENDIX I - OUTPUT SCREENSHOTS**<br>Sample Dataset<br>Outlier Removal<br>Attack Based on Protocol<br>Class Distribution<br>Flag Count<br>Service Occurences<br>Accuracy<br>Test Set Prediction<br>Home Screen<br>Normal Prediction<br>Probe Detection | 70 - 78 |
| **APPENDIX II - PROGRAM CODING** | 79 - 112 |

# ABSTRACT

The fast propagation of computer networks has changed the viewpoint of network security. An easy accessibility conditions cause computer network as susceptible against several threats from hackers.

Threats to networks are numerous and potentially devastating. Up to the moment, researchers have developed Intrusion Detection Systems (IDS) capable of detecting attacks in several available environments.

A boundlessness of methods for misuse detection as well as anomaly detection has been applied. Many of the technologies proposed are complementary to each other, since for different kind of environments some approaches perform better than others.

This project presents a new intrusion detection system that is then used to survey and classify them. The taxonomy consists of the detection principle, and second of certain operational aspects of the intrusion detection system.

In our project we have used algorithms like Naïve Bayes (NB) and Convolution Neural Network (CNN). All are measured in terms of accuracy.

# CHAPTER 1

# 1. INTRODUCTION OF PROJECT

## 1.1 Sensor network:

A network comprised of interconnected sensor nodes exchanging sensed data by wired or wireless communication.

## 1.2 Sensor node:

A device consisting of sensor(s) and optional actuator(s) with capabilities of sensed data processing and networking. Sensor node consists of a great number of nodes of the same type (sensor nodes), which are spatially distributed and cooperate with each other. Each such node has a sensing element (sensor), a microprocessor (microcontroller), which process sensor signals, a transceiver and an energy source. Distributed over the object, sensor nodes with the necessary sensors make it possible to gather information about the object and control processes which take place on this object.

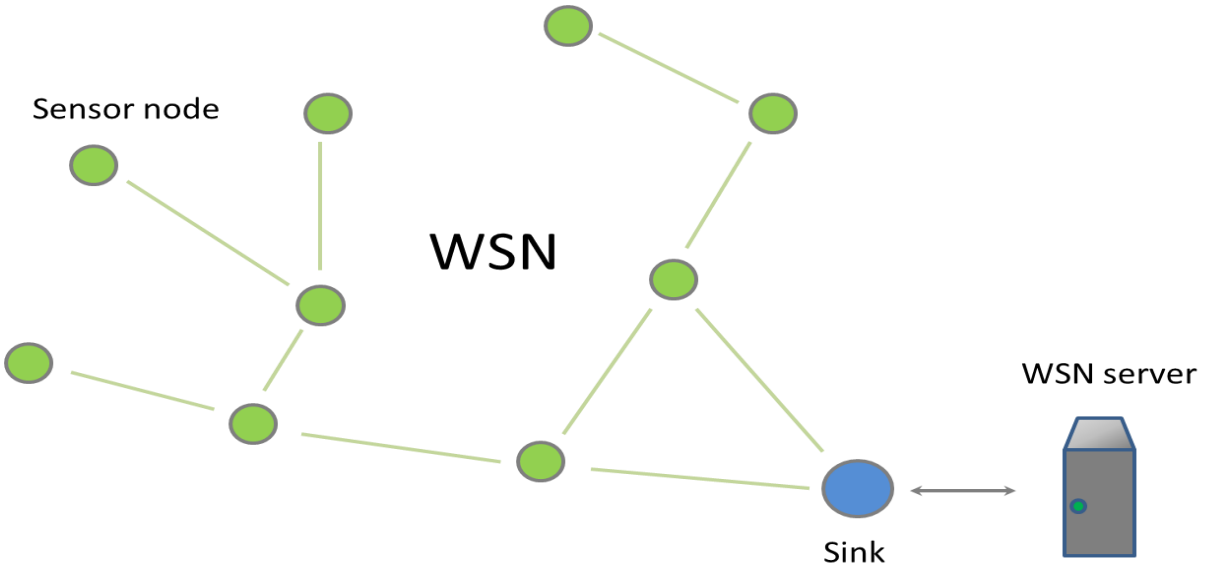



Fig 1 Overview of the network architecture

The above figure represents an example of a WSN. Here we can see a WSN which consists of twelve sensor nodes and a *network sink*, which also functions as a *gate*. Each sensor node is a device which has a transceiver, a microcontroller, and a sensitive element. Usually sensor node is an autonomous device.

Each sensor node in WSN measures some physical conditions, such as temperature, humidity, pressure, vibration, and converts them into digital data. Sensor node can also process and store measured data before transmission. Network sink is a kind of a sensor node which aggregates useful data from other sensor nodes. As a rule, network sink has a stationary power source and is connected to a *server* which is processing data received from WSN. Such connection is implemented directly, if server and WSN are placed on the same object. If it is necessary to provide a remote access to WSN, network sink also functions as a gate, and it is possible to interact with WSN through global network such as the Internet.

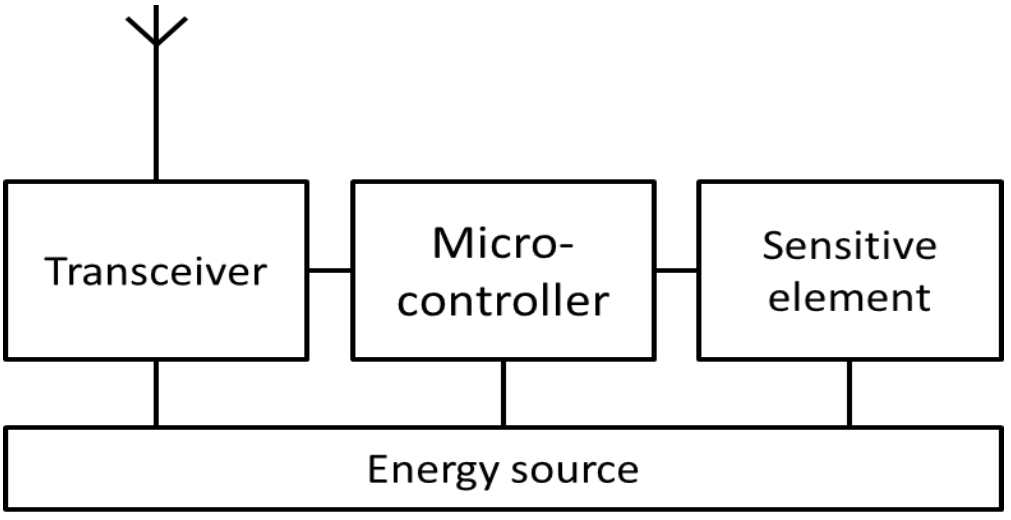



Fig 2 Sensor node inner structure

## Some Sample Applications:

• Habitat and Ecosystem Monitoring

• Seismic Monitoring

• Civil Structural Health Monitoring

• Monitoring Groundwater Contamination

• Rapid Emergency Response

• Industrial Process Monitoring

• Perimeter Security and Surveillance

• Automated Building Climate Control

## 1.3 WSN Structure:

### WSN sink:

Sensor nodes are the basis of a WSN. They collect and exchange data necessary for WSN functionality. Data collected by sensor nodes are the raw information and require processing. Depending on application, such data can be averaged statistical information or detailed measurements of parameters which define the condition of some object. A separate group of WSN applications is detecting and tracking of targets, for examples, vehicles, animals etc. Each of these cases requires processing of data provided by WSN.

Usually it is impossible to perform this processing on sensor nodes themselves, by reason of energy saving and low computing power of sensor nodes. That is why in WSNs the final part of data processing is usually made beyond sensor nodes, on WSN server. WSN server is connected to only one sensor node which is called *sink* or *base station*. Sink is a collecting point of all data in the WSN and interact both with the sensor nodes and the WSN server.

## 1.4 WSNS with the Cluster Structure:

Since energy content of sensor nodes is limited and non-renewable, it is important to use it in the most economical way. The figure illustrates the data streaming from sensor nodes to sink. Sink collects data from all the sensor nodes periodically. On the figure, every arrow between sensor nodes shows a transfer of a portion of measurements for a single period of data collection.

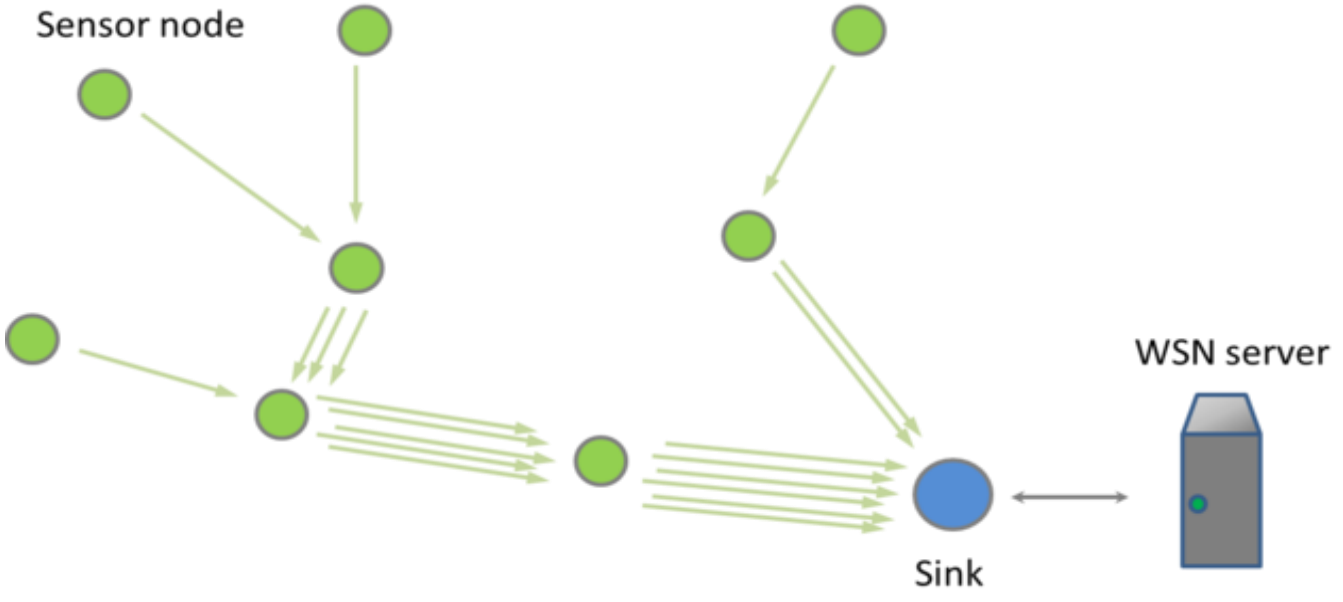



Fig 3 Data streaming from sensor nodes

Data is collected from all the sensor nodes; in result, the sensor nodes located closer to sink have to receive and transmit not only their own measurements, but also measurements from other sensor nodes which are further from sink. So, transceivers of the nearest sensor nodes retransmit much more information, and hence they consume more energy than remote sensor nodes. And since sensor nodes are usually all of the same type and have equal energy content, it leads to the fact that the nearest sensor nodes fail much earlier than remote ones, and so the former disrupt the work of the rest of WSN.

So, if WSN application provides periodical data collecting (and it happens in the most cases), it turns out that time of autonomous operating of sensor nodes which are the nearest to sink is much reduced because of more frequent retransmitting. In the long run, traffic from all the sensor nodes is going through one sensor node that is nearest to sink. And the more sensor nodes are in a WSN, the higher is this traffic. From the point of view of energy saving, big WSNs with only one sink cannot consume resources effectively.

To solve this problem, it is necessary to divide the WSN into clusters. Each cluster has its own sink and, in fact, is a separate, but smaller, WSN. And each sink communicates with the server directly. The figure represents a network with two sinks. On the figure the arrows also used to represent the amount of transmitted data. As we can see, the number of retransmissions is significantly decreased, and it reduces the load on the nearest to sinks sensor nodes.

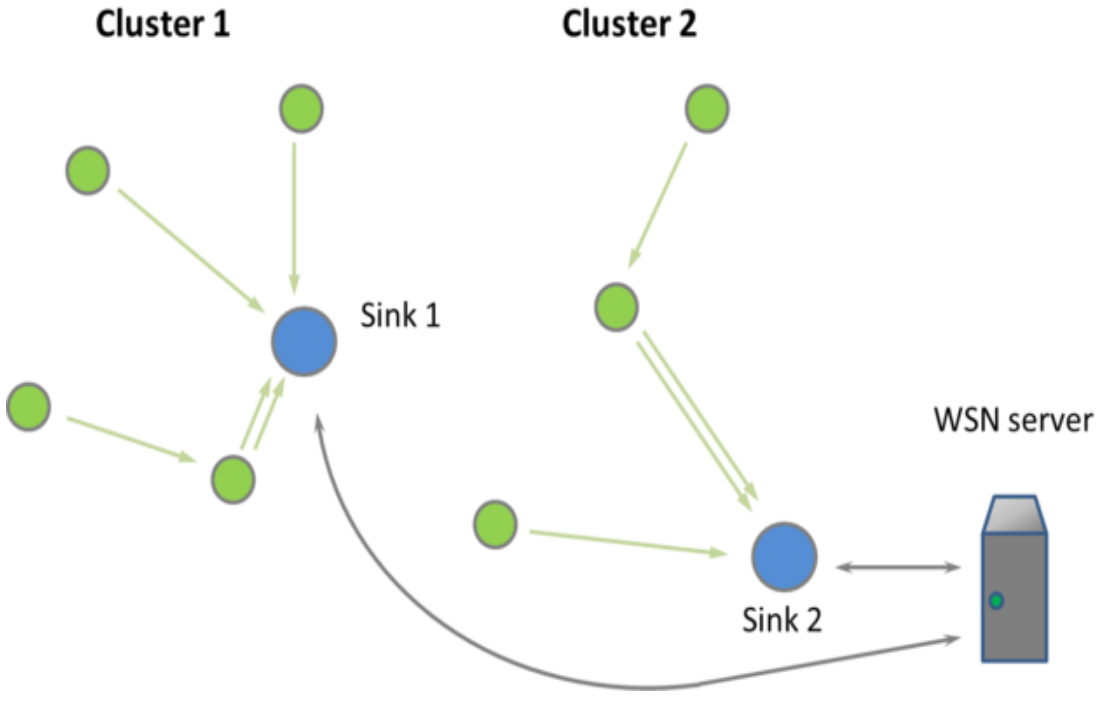



Fig 4 Multiple-sink WSN

Multiple-sink WSN is not a random division of one WSN into parts. In the most cases such division is made automatically when WSN is deployed and used. Sensor nodes automatically choose the sink to which they send data. This choice is made according to the algorithm of WSN protocol. Depending on requirements of the application, different criteria may be used, for example, the minimum time for data delivery, the minimum number of retransmissions, achieving the optimal traffic distribution in WSN and others.

## 1.5 WSN Gate:

WSN organization schemes considered above suppose placement of all WSN elements in the same location. In practice, there is often necessary to have a remote access to WSN data. For example, WSN can be deployed in woodland in suburbs, and collecting and processing of WSN data have to be done in the office in a city. To organize data transmission from WSN to a remote server one uses specialized gates which receive sensor network data from sink and retransmit them using other (i. e. non-WSN) communication standard, wired or wireless. The figure represents such a network, which transmit collected data to server through the Internet using a gate.

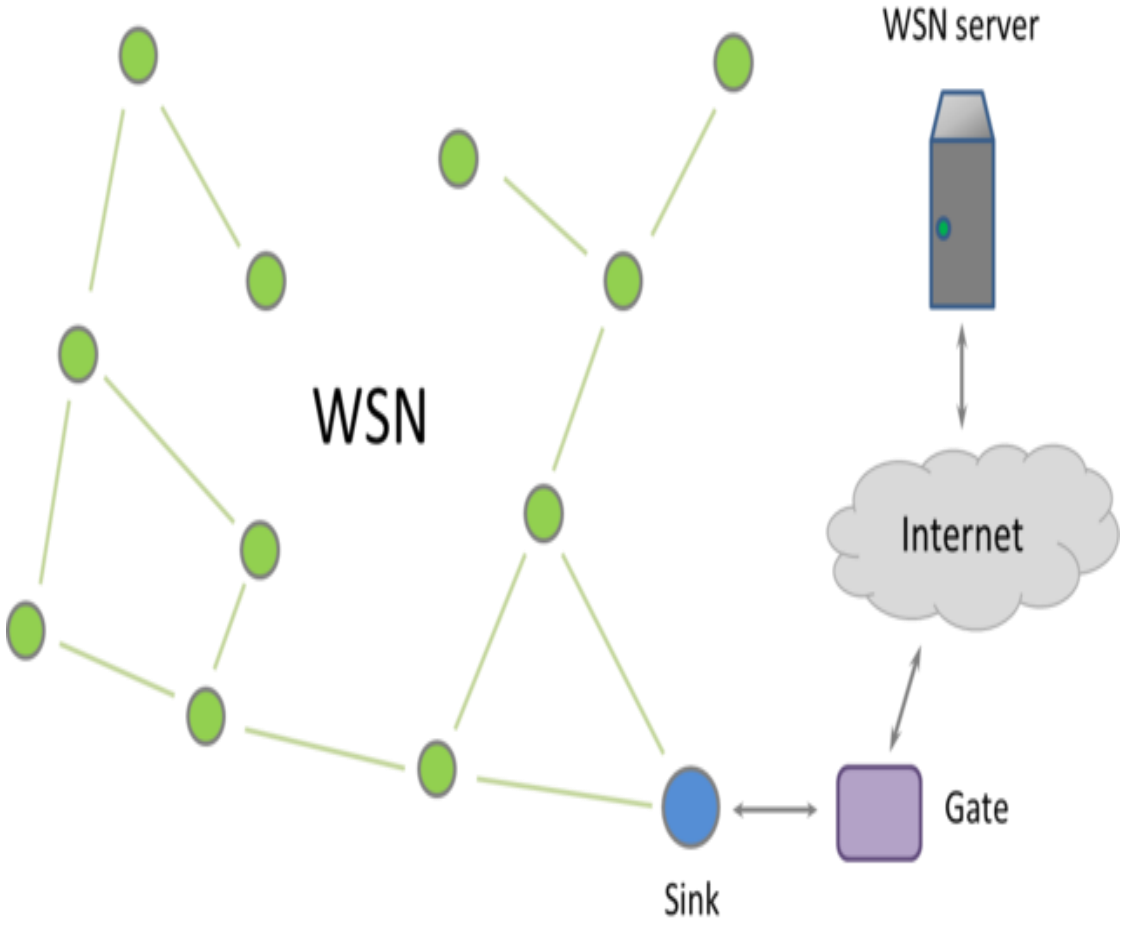



Fig 5 WSN server is connected via the Internet

Gates also provide the possibility to organize service provision. Nowadays, when access to the Internet via cellular, cable and satellite networks is available almost in any place in the world, connection of WSNs to the Internet in most cases is easy to implement. The figure represents the scheme of possible interaction between a user and a WSN.

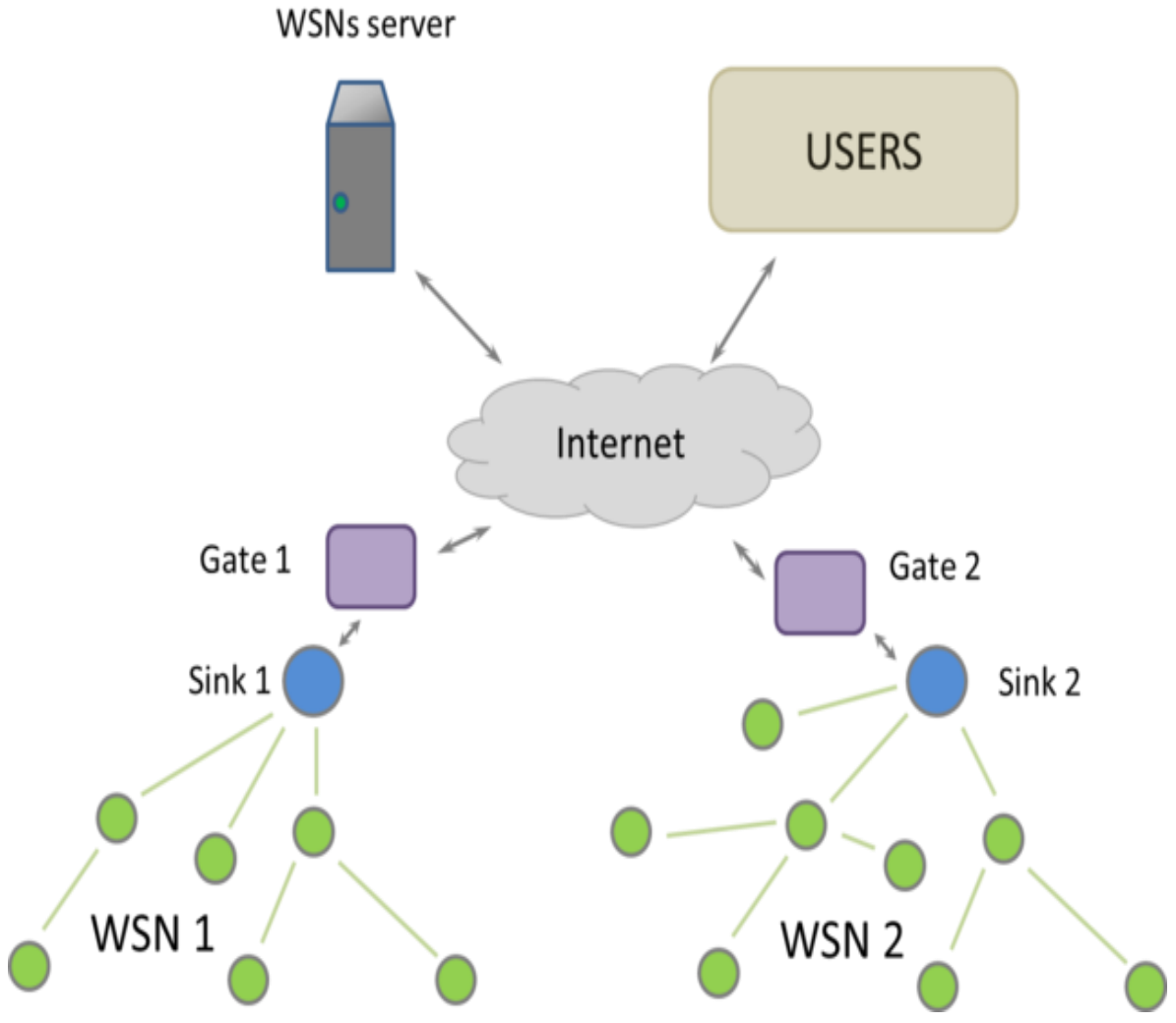



Fig 6 Scheme of provision of WSN services

## 1.6 Network Topology:

Previously we have described traditional applications of WSN for data collecting and processing. Such applications have a special feature: they have one data collecting point, namely sink. But there are also applications where sensor nodes have not only to send information to sink, but to exchange data between themselves. That is why there are different schemes of organization of interaction between sensor nodes within WSN. These schemes are called network topologies. The main types of network topologies for WSNs are: *star*, *tree* and *mesh*. Different WSN standards support different types of network topologies.

## Star:

The star topology is widely used in computer networks, so when WSN appeared, it started being used also for organization of interaction between sensor nodes. The main characteristic of the star topology is connecting of all the sensor nodes to sink directly. The figure schematically represents this topology. In such cases sensor nodes are not connected between themselves, and all interactions between sensor nodes are taking place only via sink. Disadvantage of this topology is limited number of sensor nodes in such WSN. This limitation appears because all the sensor nodes have to be placed in the vicinity of sink, in order to connect to it directly. Another limiting factor is sink's performance, i.e. the maximum number of supported connections.

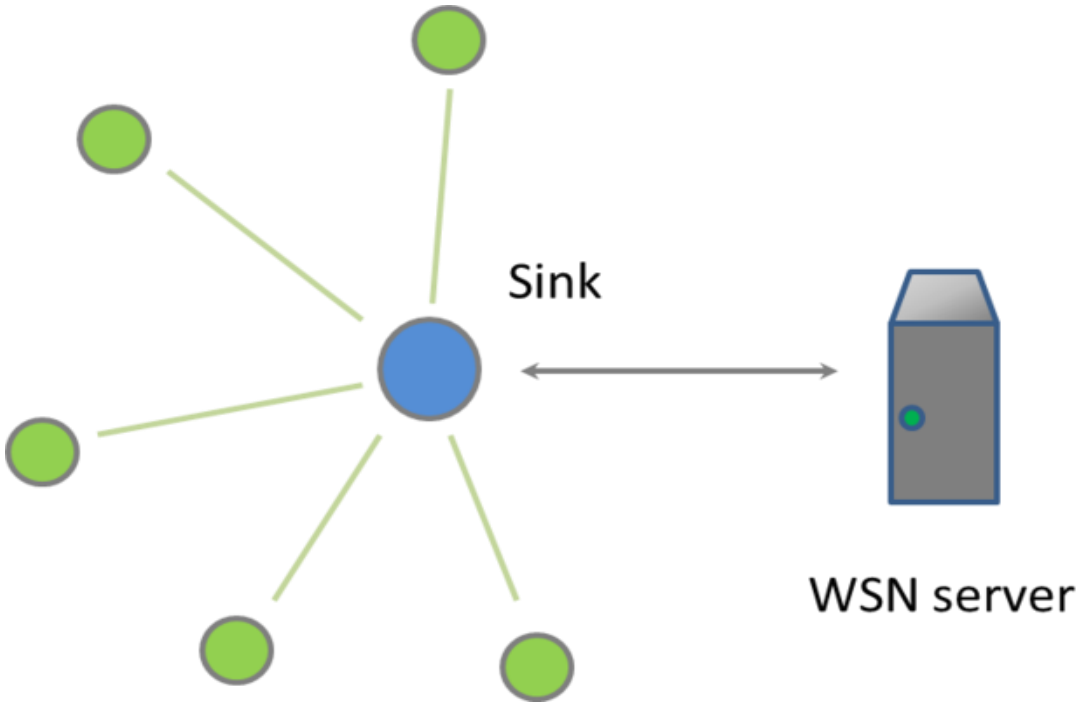



Fig 7 The star topology

## Tree:

The tree topology, in contradiction to the star topology, is much better suitable for WSN with the large number of sensor nodes. It has a hierarchical structure, as it is illustrated on the figure, Sensor nodes which are the nearest to sink interact with the sink directly. And more remote sensor nodes interact with the nearest ones according to the rules of the star topology. The tree topology also does not provide direct data exchange between all the sensor nodes. Data transmissions only from any sensor node to the sink and in the opposite direction are allowed. Also, in this topology data flow from the levels with greater numbers (i.e. "leaves") can be delivered only through the

levels with smaller numbers (i. e. “root” and “branches”). So, if on the first level there are only two sensor nodes, and the whole WSN consists of eleven sensor nodes, traffic will be delivered through these two sensor nodes much longer, because of data retransmission from nine sensor nodes on lower levels. Such network can fail quickly, because of energy consuming by the nearest to sink sensor nodes.

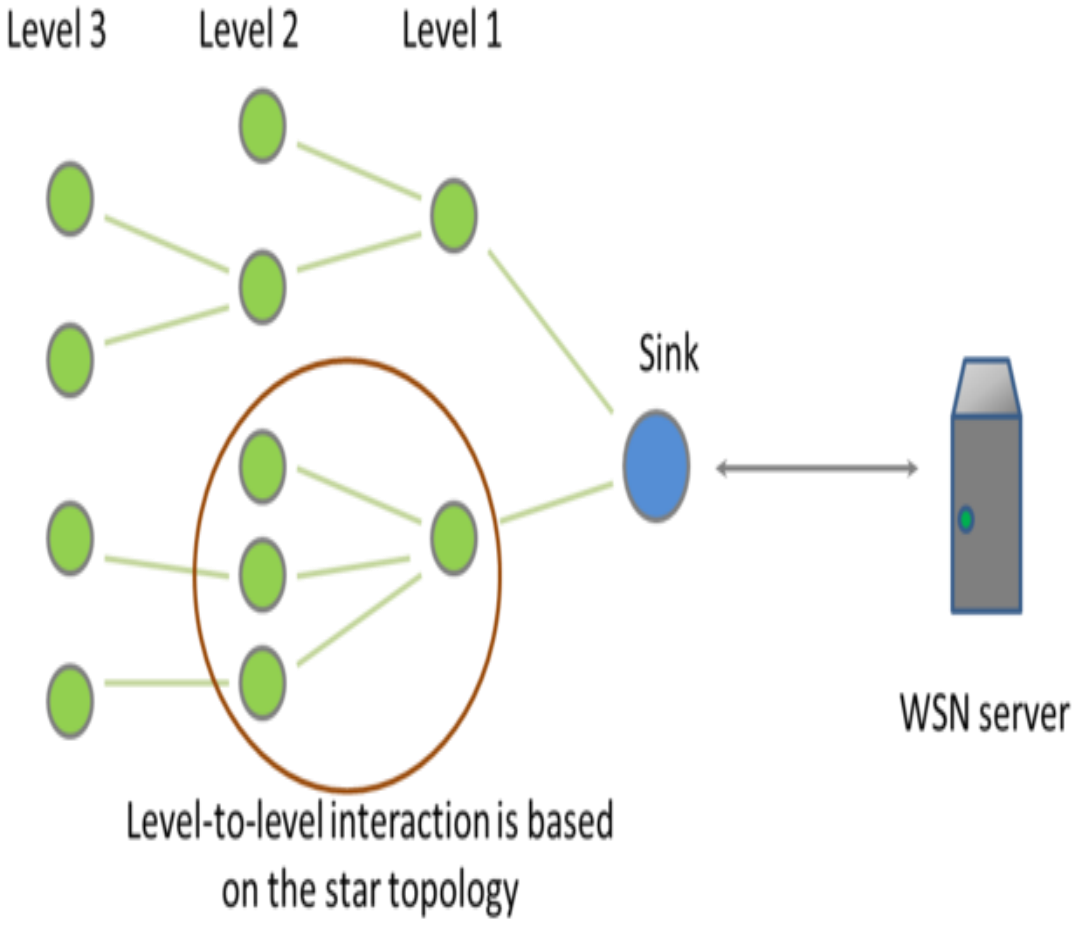



Fig 8 The tree topology

## Mesh:

The mesh topology is the most difficult one for implementation, but it provides much more opportunities for data exchange between sensor nodes. In WSN with the mesh topology interaction between sensor nodes is taking place according to the principle “with every nearest one”, as shown on The figure. It means that every sensor node cooperates with other sensor nodes, which are in its transceiver’s proximity. In such WSN data exchange between sensor nodes goes through the shortest ways and with the smallest number of retransmissions, what has a positive effect on the energy consumption of the sensor nodes.

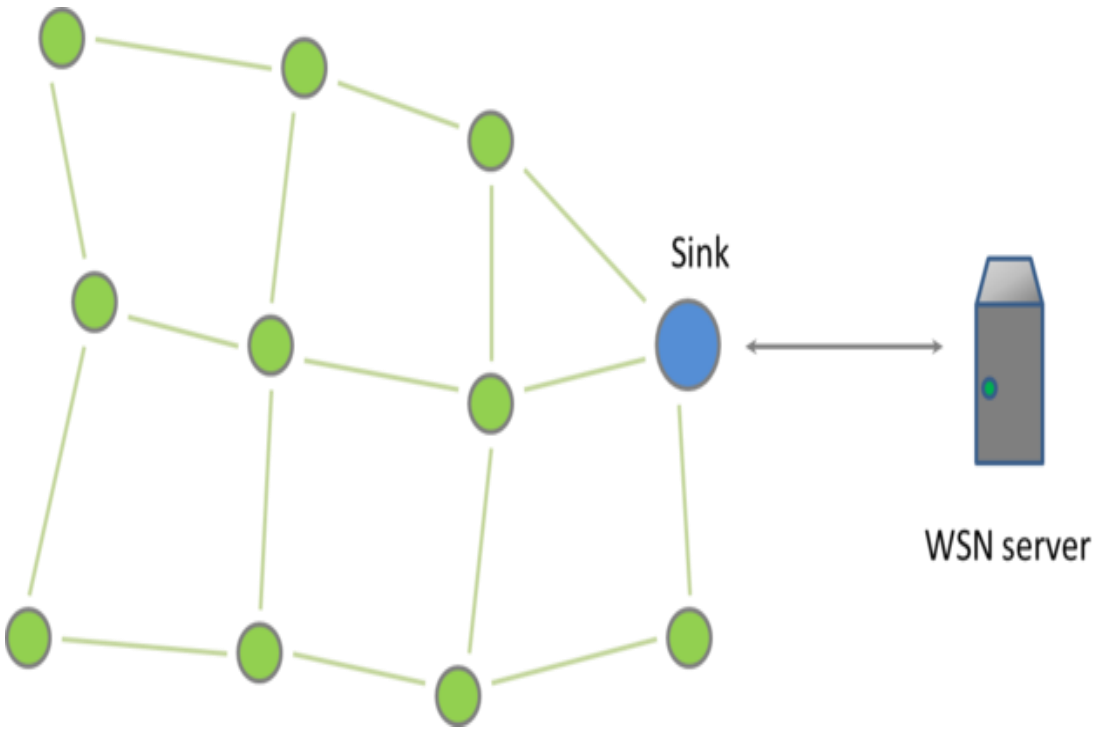



Fig 9 The mesh topology

## 1.7 General Design Issues:

One of the most important tasks which have to be solved when working out a WSN for a specific application is the choice of the hardware platform which will serve as a basis for creating a sensor node. There are a lot of sensor nodes implementations from different vendors, but all the platforms have common elements. Choosing one or other existing platform or development of a new one from scratch have to be made in order to meet WSN functional requirements. Any hardware platform provides its own set of sensor node parameters. Variety of available platforms is caused by a wide range of WSN applications, and each existing platform has its own features according to the set of the tasks it is meant for. Also we have to understand that the current level of technology makes it necessary for the researchers to constantly find a balance between such parameters as size, productivity, battery lifetime, communication range, coverage, reliability, functionality, cost etc. The figure illustrates the correlation between the primary sensor node parameters. The arrows link directly related parameters, so improving a parameter in one end of the arrow will lead to worsening of the parameter in the other end of the arrow. For example, refinement of functionality (such as increasing the number of controlled

parameters, improving the microcontroller performance) will inevitably cause an increase in cost, a decrease in battery lifetime and/or an increase in the size of a sensor node. So, the task of developing new hardware/software platforms which would support new technologies, expand the application scope, facilitate the deployment of WSNs is still relevant.

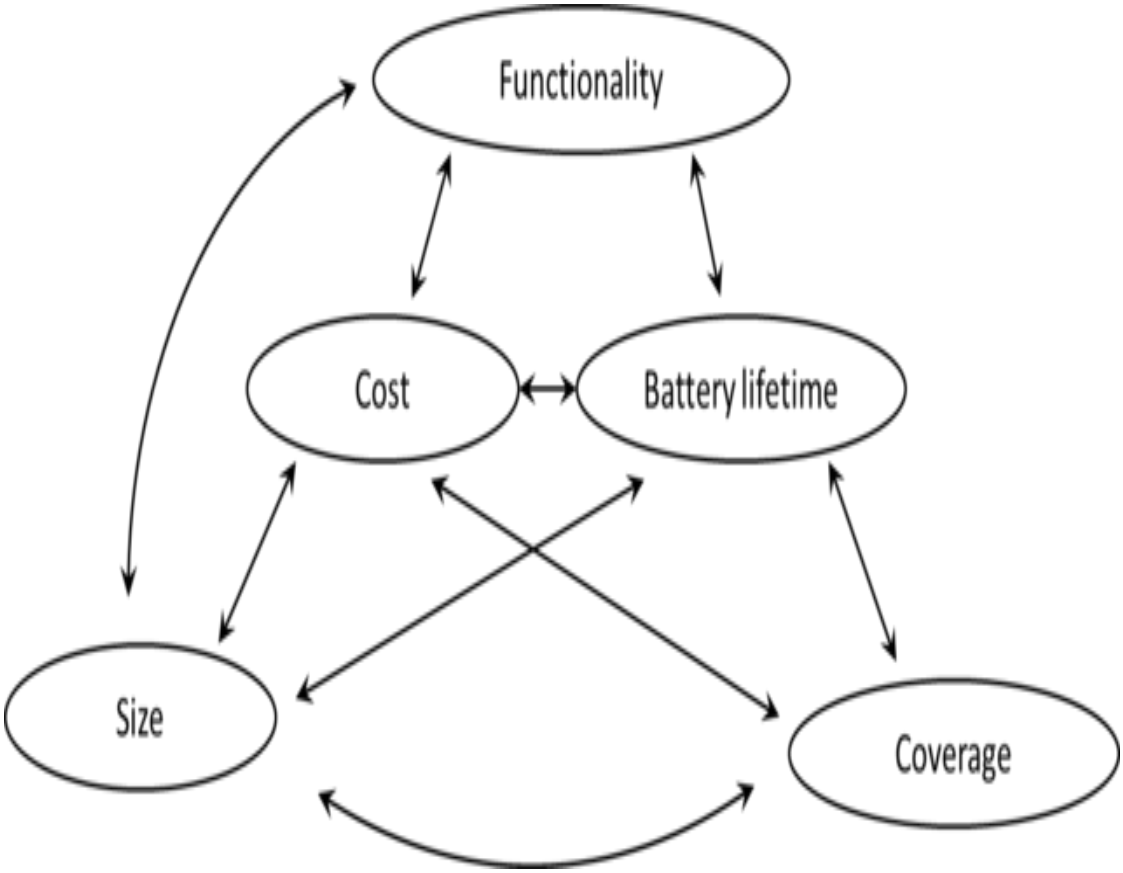



Fig 10 Relationships of The primary sensor node parameters

In the next sections we are going to consider the internal structure of a sensor node as well as the main problems of sensor node development more precisely.

## 1.8 The Key Features of Sensor Nodes:

Before starting the consideration of the sensor node's structure in more detail, we should pay more attention to the main features of sensor nodes. They directly affect the capabilities of the whole WSN. That is why the requirements made to a WSN by a concrete application can always be converted to requirements for sensor nodes.

## Energy Efficiency (Autonomy):

Unlike other common battery-powered mobile devices, sensor nodes deal with much more stringent requirements on energy efficiency, and it imposes restrictions on the all sensor node components. For example, for a mobile phone it is acceptable to keep working autonomously for a few days because the user usually, have the possibility to charge the battery if necessary. But with sensor nodes we have another situation.

The WSN parts may be spatially distributed on the area of many kilometers, especially if a WSN user is managing it via the Internet. At the same time, sensor nodes can be located in the inaccessible places, or the concrete location of each sensor node can be unknown. Also, a WSN may consist of dozens, hundreds or even thousands of sensor nodes. Under these conditions charging of sensor nodes by the user is out of question. That is why a sensor node must have high energy efficiency in order to keep working on small and inexpensive battery for a few months and even years. This ultra-low-power operation can only be achieved by using low-power hardware components.

Also, one of the key techniques extending the sensor node battery life is reduction of duty-cycle. This parameter is defined as the ratio of the sensor node active functioning time and the time when it is in the low power mode (the sleep mode). In WSNs with long lifetime sensor nodes most of the time are in the sleep mode, where sensor node power consumption is reducing in 3-4 times due to switching off all the main components excepting the part which is responsible for returning from the sleep mode when needed. After returning from the sleep mode the sensor node exchanges data with surrounding sensor nodes, takes readings from sensing elements, and then the sleep mode in turned on again.

## Platform Flexibility:

The majority of real applications require flexibility and adaptability of the WSN platform. In one application a user may need a WSN able to keep working for a few years, and herewith data update speed and data transmission delay won’t play a significant role. For example, to monitor the soil temperature and humidity there is no need of frequent readings update and fast data transmission (because the soil temperature cannot change quickly), but it is

very important that WSN which performs these functions keeps working as long as possible. In other applications such as monitoring of the spread of forest fires, fast detecting and fast data transmission will be more important, and the WSN lifetime will be less important parameter. So, each sensor node platform must have ability to be adjusted to meet the requirements of a specific application.

## Reliability:

Certainly, every WSN developer and manufacturer is interested in cost reduction of sensor nodes taking into consideration that every WSN has a great number of sensor nodes. Nevertheless, each concrete sensor node has to be reliable to such extent that it could work without breaking from the moment of turning on until the complete using of battery supply. In addition to increasing reliability of each sensor node, to provide the whole WSN reliability one may use adaptive protocols of data transmission management (*adaptive routing*). They are meant for providing WSN general robustness when certain sensor nodes are failing.

For example, if traffic from one or a few sensor nodes is going through the other sensor node and it suddenly fails, as it is illustrated on The figure, the WSN will change its structure and reconnect the "lost" node through the others nearest to it. It is worth mentioning that the main modern WSN platforms support this function.

There is also another common threat to WSN reliability which doesn't deal with reliability of any concrete sensor node. It is interference with the signals of other wireless networks and household or industrial devices' radiation. WSNs are often fully or partially located in places with significant electromagnetic fields of other wireless connection systems and appliances. In such cases these electromagnetic fields interfere with low-power transmitters in sensor node.

This interference can be significant if it falls on radio spectrum in operation frequency range of sensor nodes' transmitters. In this case connection between nodes in the interference area can get much worse or even break down, and here even operable sensor nodes cannot transmit collected data. In such situations to increase the system's robustness to a node failure, a wireless sensor network must also be robust to external interference. The robustness of wireless links can be greatly increased through the use of *multi-channel and spread spectrum radios*. The

figure represents principal of operation of the sensor nodes which support multi-channel radios. So, the possibility to change frequency channel for data transmitting is a necessary function for WSNs that are supposed to be deployed in a harsh electromagnetic environment.

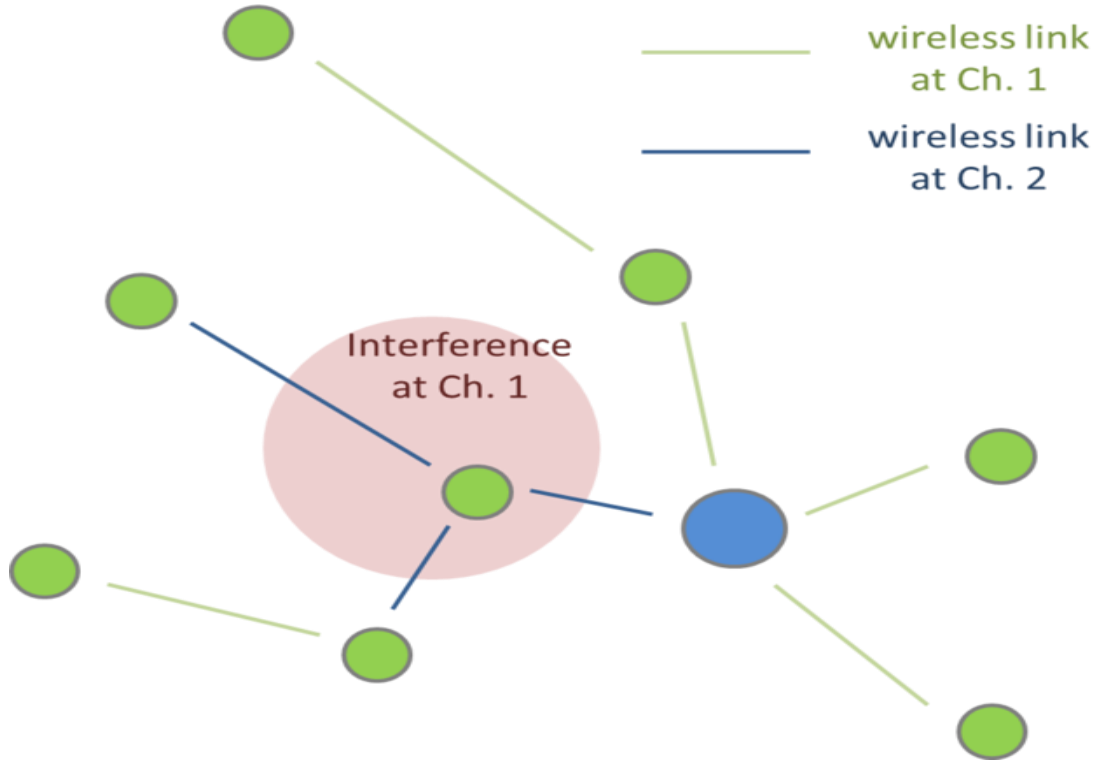



Fig 11 WSN working under conditions of strong interference on communication channel1

## Information Security:

Certain WSN applications make stringent requirements to information security. And this requirement becomes increasingly important, by reason of growth of cybernetic threats when WSNs are connected to the Internet. In order to meet the security requirements, sensor nodes must be capable of performing complex encrypting and authentication algorithms. In fact, radio communication channels can be easily tapped and become available for intruders. The only way to avoid it is encrypting of all data transmitted in the WSN. Many modern sensor nodes make it possible to flexibly set traffic encryption in the network. In some platforms it is made by means of software, but some sensor nodes include special hardware encryption blocks. But in any case, encryption requires additional expenditure of energy, and it has negative impact on WSN lifetime.

Another aspect of information security in WSNs is protection of sensor nodes' internal memory. Sensor node internal memory includes not only information meant to be transmitted in the WSN, but also private keys for traffic encryption. So it must be reliably protected from external intervention.

These information security aspects have to be taken into account simultaneously. On the one hand, weak protection of internal memory will make WSN private keys available making it possible to "crack" the WSN no matter how complex encryption algorithm is. On the other hand, weak traffic encryption will make data transmitted in the network available for sniffing and alteration by the intruder, even if internal memory of each sensor node is well protected.

The security issues in WSNs will be considered in detail in Section 6.3.

## Transceiver Performance:

One of the key sensor node characteristics is transceiver performance. The main parameters of transceiver performance which affect the sensor node characteristics are maximum data transfer rate, frequency range, modulation method, receiver sensitivity and transmitter power.

All these sensor node technical parameters affect such main WSN characteristic as reliability, the minimum spatial density of sensor nodes, the maximum readings update rate and lifetime. So, sensor node transceiver parameters are one of the main characteristics of WSN platform.

Above we have considered how the interference affects WSN reliability on a qualitative level. It is possible to estimate quantitatively how interference affects wireless link between sensor nodes with the help of the mentioned transceiver characteristics.

For estimating the impact of noise on the quality of signal reception, in information theory the *signal-to-noise ratio* (SNR) is used. This ratio shows in how many times the wanted signal (signal from other sensor node) received by the sensor node receiver exceeds the power level of interference.

The higher the SNR is, the more powerful is useful signal as compared with noise, and the higher is probability to receive the signal correctly. For every method of signal modulation there is a special SNR value at which or above which communication between receiver and transceiver is possible. Also, in information theory there is a fundamental principle, which can be

expressed (in a simplified form) by the following statement: the higher SNR is, the higher is maximum data transfer rate.

Now it is obvious that the SNR affects reliability and quality of wireless link between two sensor nodes. And the higher SNR is, the better is the quality of communication. So, the more powerful is emission of the first sensor node's transmitter, the higher is SNR of receiving sensor node, and the higher is wireless link quality, hence the whole WSN reliability.

In addition, the closer the sensor nodes to each other are located, the better is the SNR for both of them. It means that the maximum distance between sensor nodes in WSN is inextricably linked with power of sensor node transmitter, and the maximum distance between sensor nodes specifies the minimum number of sensor nodes necessary for covering the given space by WSN.

Sensor node receiver *sensitivity* represents the ability to receive weak signals, for example, at a great distance from other sensor nodes, and it, as well as transmitter power, affects the maximum distance between the sensor nodes. It is worth mentioning that increasing the power and sensitivity of sensor node transmitter and receiver leads to higher energy consumption and cost of the sensor nodes.

But this dependence is not linear, and the benefit in increasing the range of sensor node is not so great. That is why the most common characteristics of transceivers measure up with one mw of power, which is acceptable in terms of energy consumption and provides reliable wireless connection between sensor nodes at the distance of about 10 meters.

Frequency range of transceiver affects the maximum possible rate of data exchange and the maximum possible distance between the sensor nodes. At the heart of this dependence are the physical laws of the radio signal. According to these laws, the higher is frequency used as carrier, the stronger is the signal attenuation with the distance.

That is why the sensor nodes platforms which operate in lower frequency ranges allow having higher value of the maximum distance between sensor nodes in WSN. But the basic physical laws don't allow using very low frequencies for connecting sensor nodes in the majority of WSNs, because the size of the transceiver's antenna has to be the bigger, the lower is the frequency, and it affects the size of the sensor nodes.

The maximum speed of data transmission and reception by sensor node transceiver restrains the maximum speed of data gathering in the WSN. In addition, the higher is the maximum speed of data transmission, the higher is the energy consumption of transceiver during transmission and reception. On the other hand, the higher is speed of transmission, the less time is necessary for transmitting the same data; hence, transceiver will be switched on for less time. But high speed of transmitting also requires more computing power and energy for this computing, which is not always acceptable.

## 1.9 Characteristics of the WSN which utilizes such sensor nodes computing power:

Sensor node's microcontroller (and hence, consumes battery energy) uses its computing powers for two kinds of tasks. First kind of these tasks deals with supporting WSN functioning, the second task is reading and processing measurements of sensing element. Both kinds of tasks require certain computing power and take the time of the microcontroller. When the micro controller is busy, its energy consumption becomes significant.

The task of supporting WSN functioning, in the first place, is implementation data reception and further transmission algorithms that are part of the WSN communication protocol. Every sensor node is permanently receiving data from other surrounding sensor nodes. Microcontroller identifies these data and depending on the content transmits to the nearest sensor nodes, ignores them or saves to internal memory for further processing. All it happens in accordance with the WSN communication protocol.

Computing power of sensor node microcontroller has to be the higher, the higher is the maximum rate of data exchange, so that to have time for data decoding.

We can see the same situation with computing powers necessary for reading and processing of sensor measurements. Sensitive elements can produce a plenty of data which have to be timely processed. And the types of necessary processing can vary a lot, from simple averaging, digital filtration, tracking of some threshold exceeding to calculation of autocorrelation and spectral analysis. The last two operations are the example of the especially resources-consuming ones.

## Size and Cost:

Miniaturization, price reduction, and improvement of other parameters are the most important priorities from the very first researches in WSNs. The good example is the SmartDust project which took place in the end of 1990s and the beginning of 2000s. Miniaturization and price reduction of sensor were constantly expanding the possibilities of WSN applications, and in future they can lead to the widespread use of WSNs and to uprising of ubiquitous WSNs.

Above we have already considered the dependence between different WSN characteristics, and now, after considering the additional characteristics, it is possible to imagine how difficult is to find balance between them when developing the sensor nodes.

## 1.10 Introduction of Domain:

## Machine Learning:

In the statistical context, Machine Learning is defined as an application of artificial intelligence where available information is used through algorithms to process or assist the processing of statistical data. While Machine Learning involves concepts of automation, it requires human guidance. Machine Learning involves a high level of generalization in order to get a system that performs well on yet unseen data instances.

Machine learning is a relatively new discipline within Computer Science that provides a collection of data analysis techniques. Some of these techniques are based on well-established statistical methods (e.g. logistic regression and principal component analysis) while many others are not.

Most statistical techniques follow the paradigm of determining a particular probabilistic model that best describes observed data among a class of related models. Similarly, most machine learning techniques are designed to find models that best fit data (i.e. they solve certain optimization problems), except that these machine learning models are no longer restricted to probabilistic ones.

Therefore, an advantage of machine learning techniques over statistical ones is that the latter require underlying probabilistic models while the former do not. Even though some machine learning techniques use probabilistic models, the classical statistical techniques are most often too stringent for the oncoming Big Data era, because data sources are increasingly complex and multi-faceted. Prescribing probabilistic models relating variables from disparate data sources that are plausible and amenable to statistical analysis might be extremely difficult if not impossible.

Machine learning might be able to provide a broader class of more flexible alternative analysis methods better suited to modern sources of data. It is imperative for statistical agencies to explore the possible use of machine learning techniques to determine whether their future needs might be better met with such techniques than with traditional ones.

## 1.11 Objective of the Problem:

The goal of intrusion detection is to monitor the network assets to detect anomalous behavior and misuse in network. Intrusion detection concept was introduced in early 1980's after the evolution of internet with surveillance end monitoring the threat. There was a sudden rise in reputation and incorporation in security infrastructure. Since then, several events in IDS technology have advanced intrusion detection to its current state. James Anderson's wrote a paper for a government organization and imported an approach that audit trails contained important information that could be valuable in tracking misuse and understanding of user behavior.

Then the detection appeared and audit data and its importance led to terrific improvements in the subsystems of every operating system. IDS and Host Based Intrusion Detection System (HIDS) were first defined. In 1983, SRI International and Dorothy Denning began working on a government project that launched a new effort into intrusion detection system development. Around 1990s the revenues are generated and intrusion detection market has been raised. Real secure is an intrusion detection network developed by ISS. After a year, Cisco recognized the priority for network intrusion detection and purchased the Wheel Group for attaining the security solutions. The government actions like Federal Intrusion Detection Networks (FID Net) were designed under Presidential Decision Directive 63 is also adding impulse to the IDS.

The main objectives of the problem are as follows:

1. Less throughput.
2. No energy checkup after each communication.
3. Time delay is high.
4. High packet loss.
5. Delivery ratio is less.
6. Attacking is very high.

## 1.12 Scope of the project:

We propose a method to detect intrusions in computer network security. The main idea is to reuse the already available system information that is generated at various layers of a network stack. To the best of our knowledge, this is the first such approach for intrusion detection in computer network security. Wireless Sensor Network consists of large number of nodes, which will be in distributed nature. Security is a very important consideration while designing a Wireless Sensor Network. So, an Advanced Intrusion Detection System has been proposed where the Heterogeneous Hybrid Intrusion Detection System (H-HIDS), Intrusion Detection Prediction based are implemented in various stages in order to assure maximum possible security from the Intrusions.

The main scope of this project is to provide communication continuously by checking the energy at every communication and also to find out the intrusion that is happening inside the network. The main objectives of the projects are as follows:

1. To find out early attacks.
2. To minimize packet loss.
3. More throughputs.
4. To reduce time consumption.
5. Continuous energy check-up of all nodes to avoid node failure.

# CHAPTER 2

# 2. LITERATURE REVIEW

## 2.1 NL-IDS: Trust Based Intrusion Detection System for Network layer in Wireless Sensor Networks Umashankar Ghugar, Jayaram Pradhan IEEE 2023:

From the last few years, security in wireless sensor network (WSN) is essential because WSN application uses important information sharing between the nodes.

There are large number of issues raised related to security due to open deployment of network. The attackers disturb the security system by attacking the different protocol layers in WSN. The standard AODV routing protocol faces security issues when the route discovery process takes place.

The data should be transmitted in a secure path to the destination. Therefore, to support the process we have proposed a trust based intrusion detection system (NL-IDS) for network layer in WSN to detect the Black hole attackers in the network.

The sensor node trust is calculated as per the deviation of key factor at the network layer based on the Black hole attack. We use the watchdog technique where a sensor node continuously monitors the neighbor node by calculating a periodic trust value.

Finally, the overall trust value of the sensor node is evaluated by the gathered values of trust metrics of the network layer (past and previous trust values). This NL-IDS scheme is efficient to identify the malicious node with respect to Black hole attack at the network layer.

To analyze the performance of NL-IDS, we have simulated the model in MATLAB R2015a, and the result shows that NL-IDS is better than Wang et al. [11] as compare of detection accuracy and false alarm rate.

## 2.2 Analyzing the Vulnerability of Wireless Sensor Networks to a Malicious Matched Protocol Attack George D. O'Mahon, Philip J. Harris, Colin C. Murphy IEEE 2023:

Safety critical, Internet of Things (IoT) and space-based applications have recently begun to adopt wireless networks based on commercial off the shelf (COTS) devices and standardized protocols, which inherently establishes the security challenge of malicious intrusions. Malicious intrusions can cause severe consequences if undetected, including, complete denial of services. Particularly, any safety critical application requires all services to operate correctly, as any loss can be detrimental to safety and/or privacy.

Therefore, in order for these safety critical services to remain operational and available, any and all intrusions need to be detected and mitigated. Whilst intrusion detection is not a new research area, new vulnerabilities in wireless networks, especially wireless sensor networks (WSNs), can be identified. In this paper, a specific vulnerability of WSNs is explored, termed here the matched protocol attack.

This malicious attack uses protocol-specific structures to compromise a network using that protocol. Through attack exploration, this paper provides evidence that traditional spectral techniques are not sufficient to detect an intrusion using this style of attack. Furthermore, a ZigBee cluster head network, which co-exists with ISM band services, consisting of XBee COTS devices is utilized, along with a real time spectrum analyzer, to experimentally evaluate the effect of matched protocol interference on a realistic network model. Results of this evaluation are provided in terms of device errors and spectrum use. This malicious challenge is also examined through Monte-Carlo simulations.

A potential detection technique, based on coarse inter-node distance measurements, which can theoretically be used to detect matched protocol interference and localize the origin of the source, is also suggested as a future progression of this work. Insights into how this attack style preys on some of the main security risks of any WSN (interoperability, device

## 2.3 Abnormal-Node Detection Based on Spatio-Temporal and Multivariate-Attribute Correlation in Wireless Sensor Networks Nesrine Berjab, Hieu Hanh Le, Chia-Mu Yu, Sy-Yen Kuo, Haruo Yokota IEEE 2023:

In wireless sensor networks (WSNs), data can be subject to malicious attacks and failures, leading to unreliability. This vulnerability poses a challenge to environmental monitoring applications by creating false alarms. To guarantee a trustworthy system, we therefore need to detect abnormal nodes. In this paper, we propose a new framework for detecting abnormal nodes in clustered heterogeneous WSNs.

It makes use of observed spatiotemporal (ST) and multivariate-attribute (MVA) sensor correlations, while considering the background knowledge of the monitored environment. Based on the ST correlations, the collected data is analyzed by computing the cross correlation between sensor streams. A new method is proposed for evaluating the intensity of the correlation between two sensor streams.

The cross correlation value obtained is compared against two thresholds, the lag threshold and the correlation threshold. Based on available background knowledge and the observed MVA correlations, a number of rules are presented to detect abnormal nodes while identifying real events. Our experiments on real-world sensor data demonstrate that our approach captures the correlation and discovers abnormal nodes efficiently.

## 2.4 Secure Knowledge and Cluster-Based Intrusion Detection Mechanism for Smart Wireless Sensor Networks Amjad Mehmood, Akbar Khanan, Muhammad Muneer Umar, Salwani Abdulla, Khairul Akram Zainol Ariffin, Houbing Song IEEE 2023:

Wireless sensor networks, due to their nature, are more prone to security threats than other networks. Developments in WSNs have led to the introduction of many protocols specially developed for security purposes. Most of these protocols are not efficient in terms of putting an

excessive computational and energy consumption burden on small nodes in WSNs. This paper proposes a knowledge-based context-aware approach for handling the intrusions generated by malicious nodes. The system operates on a knowledge base, located at the base station, which is used to store the events generated by the nodes inside the network.

The events are categorized and the cluster heads (CHs) are acknowledged to block maliciously repeated activities generated. The CHs can also get informational records about the maliciousness of intruder nodes by using their inference engines. The mechanism of events logging and analysis by the base station greatly affects the performance of nodes in the network by reducing the extra security-related load on them.

## 2.5 Threshold Tuning-Based Wearable Sensor Fault Detection for Reliable Medical Monitoring Using Bayesian Network Model Haibin Zhang, Jiajia Liu, Nei Kato IEEE 2023:

As the medical body sensor network (BSN) is usually resource limited and vulnerable to environmental effects and malicious attacks, faulty sensor data arise inevitably which may result in false alarms, faulty medical diagnosis, and even serious misjudgment. Thus, faulty sensory data should be detected and removed as much as possible before being utilized for medical diagnosis-making. Most available works directly employed fault detection schemes developed in traditional wireless sensor network (WSN) for body sensor fault detection.

However, BSNs adopt a very limited number of sensors for vital information collection, lacking the information redundancy provided by densely deployed sensor nodes in traditional WSNs. In light of this, a Bayesian network model-based sensor fault detection scheme is proposed in this paper, which relies on historical training data for establishing the conditional probability distribution of body sensor readings, rather than the redundant information collected from a large number of sensors.

Furthermore, the Bayesian network-based scheme enables us to minimize the inaccuracy rate by optimally tuning the threshold for fault detection. Extensive online dataset has been

adopted to evaluate the performance of our fault detection scheme, which shows that our scheme possesses a good fault detection accuracy and a low false alarm rate.

## 2.6. A Light-Weight Countermeasure to Forwarding Misbehavior in Wireless Sensor Networks: Design, Analysis, and Evaluation Cong Pu, Sunho Lim IEEE 2023:

Due to the lack of centralized coordination, physical protection, and security requirements of inherent network protocols, wireless sensor networks (WSNs) are vulnerable to diverse denial-of-service (DoS) attacks that primarily target service availability by disrupting network routing protocols or interfering with on-going communications. In this paper, we propose a light-weight countermeasure to a selective forwarding attack, called SCAD, where a randomly selected single checkpoint node is deployed to detect the forwarding misbehavior of malicious node.

The proposed countermeasure is integrated with timeout and hop-by-hop retransmission techniques to quickly recover unexpected packet losses due to the forwarding misbehavior or bad channel quality. We also present a simple analytical model and its numerical result in terms of false detection rate. We conduct extensive simulation experiments for performance evaluation and comparison with the existing CHEMAS and CAD schemes. The simulation results show that the proposed countermeasure can improve the detection rate and packet delivery ratio (PDR) as well as reduce the energy consumption, false detection rate, and successful drop rate.

## 2.7 Energy Efficient Detection-Removal Algorithm for Selective Forwarding Attack in Wireless Sensor Networks T.R Sreelakshmi, G.S Binu IEEE 2023:

Wireless sensor networks (WSNs) propose the promise of a flexible, low cost solution for monitoring critical infrastructure. Sensor networks have been recommended for applications such as traffic monitoring, military and battlefield surveillance. Wireless sensor networks are more prone to security attacks due to their broadcasting nature of the transmission medium and

unattended deployment of nodes in hostile and unfriendly areas where they are not protected as compared to wired networks. Attackers can deploy various types of security attacks to obstruct the security of WSNs. Network layer attacks are more severe since if the routing information is disregarded, disturbances may bring about routing loops, changing of routes etc. Selective forwarding attack is a type of active attack affecting network layers that selectively drops or refuses to forward the data packets.

This paper discusses about an energy efficient detection-removal algorithm for effective detection of selective forwarding attack in a clustered WSN scenario. The impact of the malicious node in network parameters like packet delivery ratio, throughput, residual energy of network and end to end delay are analyzed.

## 2.8 False Data Injection Prevention in Wireless Sensor Networks using Node-level Trust Value Computation B. Sreevidya, M. Rajesh IEEE 2023:

Wireless Sensor Networks are extensively used in developing applications for surveillance, habitat monitoring, border security, intrusion detection etc. Most of these applications require secure data transmission among the nodes of the network. Out of the different types of attacks a data critical application faces, False Data Injection attacks are the most damaging one. So prevention of False Data Injection attacks is a crucial aspect while building data critical wireless sensor network applications.

Researchers have suggested cryptographic schemes like RSA, ECC for the prevention of False Data Injection Attacks. Use of cryptographic techniques increases the computation complexity on all the nodes and the energy constraints on WSN demands an alternate solution for False Data Injection attacks prevention. The proposed work aims on using trust parameter of every node to distinguish malicious and non-malicious nodes and use only trusted nodes to forward the packet to destination thus by prevention FDI attacks. Simulation is carried out with the help of Network Simulator 2 (NS2). The results show the energy consumption is less in the proposed scheme compared to the cryptographic technique.

## 2.9 A Comprehensive Trust-Aware Routing Protocol with Multi-Attributes for WSNs Boyuan Sun, Donghui Li IEEE 2023:

Due to the impact of an open deployment environment, severe restrictions in power with poor hardware equipment, and a lack of centralized administration in management, wireless sensor networks (WSNs) are extremely vulnerable to malicious attacks aimed at routing and other aspects. To face this problem, we propose a novel trust-aware routing protocol for WSNs which incorporates multi attributes (TRPM) of sensor nodes in terms of communication, data, energy, and recommendation. The proposed trust model relies on an improved sliding time window considering attack frequency to facilitate the discovery of malicious behaviors of attackers. Combined with effective routing detection and maintenance protocol, the performance of our solution is tested through a wide set of simulation experiments. Extensive results reveal that an average packet transfer rate of TRPM is increased by about 19% and time consumption on the routing update is shortened by about 11%.

## 2.10 Heterogeneous statistical QoS provisioning over 5G mobile wireless networks Author: Xi Zhang ;Wenchi Cheng ; Hailin Zhang Published in: IEEE Network ( Volume: 28, Issue: 6, Nov.-Dec. 2023 ):

in this article we propose a novel heterogeneous statistical QoS provisioning architecture for 5G mobile wireless networks. First, we develop and analyze the new heterogeneous statistical QoS system model by applying and extending the effective capacity theory. Then, through the wireless coupling channels, we apply our proposed heterogeneous statistical QoS architecture to efficiently implement the following powerful 5G-candidate wireless techniques: 1) device-to-device networks; 2) full-duplex networks; and 3) cognitive radio networks, respectively, for providing heterogeneous statistical delay-bounded QoS guarantees. Finally, using the simulation experiments we show that our proposed architecture and schemes significantly outperform the existing traditional statistical delay-bounded QoS provisioning schemes in terms of satisfying the heterogeneous delay-bounded QoS requirements while maximizing the aggregate system throughput over 5G mobile wireless networks.

# CHAPTER 3

# 3. PROBLEM STSTEMENT AND METHODOLOGY

## 3.1 Problem Definition:

An IDS is referred as burglar alarm. For example, the lock system in the house protects the house from theft. But if somebody breaks the lock system and tries to enter into the house, it is the burglar alarm that detects that the lock has been broken and alerts the owner by raising an alarm. Moreover, Firewalls do a very good job of filtering the incoming traffic from the Internet to circumvent the firewall. For example, external users can connect to the Intranet by dialing through a modem installed in the private network of the organization; this kind of access cannot be detected by the firewall.

An Intrusion Prevention System (IPS) is a network security/threat prevention technology that audits network traffic flows to detect and prevent vulnerability exploits. There are two types of prevention system they are Network (NIPS) and Host (HIPS). These systems watch the network traffic and automatically take actions to protect networks and systems. IPS issue is false positives and negatives.

False positive is defined to be an event which produces an alarm in IDS where there is no attack. False negative is defined to be an event which does not produces an alarm when there is an attacks takes place. Inline operation can create bottlenecks such as single point of failure, signature updates and encrypted traffic. The actions occurring in a system or network is measured by IDS.

## 3.2 Methodology:

The IDS have been implemented in organizations to collect and analyze various types of attacks within a host system or a network. In addition, to identify and detect possible threats violations, which involve both intrusions, which are the attacks from outside the organizations and misuses that are known as the attacks within the organizations. In this paper, we proposed the integrated model which involves a combination of the two systems Intrusion Detection (ID) and Intrusion Prevention (IP) adding to those getting benefits from well-known techniques: intruder Detection

(ID) which is totally different from most of the recent works that focused only on using one system, either detection or prevention and also using either Intruder detection or Signature based detection. Some works even used a hybrid method which is a combination of both such as the work presented where the researchers used ID based on Signature but even then, their method was not provided with prevention capabilities.

On the other hand, in our case, we proposed to use our approach IDPS, which not only can detect the attack but also can further stop it using the capabilities of prevention system, which has not been utilized in the previous works. Therefore, the proposed system can outperform the hybrid system in terms of preventing the attack from conducting any bad action through blocking the event and saving that threat with the other signatures in order to be observed by Signature Based Intrusion Detection for next time so that it can be detected earlier.

Finally, deploying such integrated model in the Wireless environment will reduce the probability of risks than the normal system or even than other systems, which are just provided with Intrusion Detection methods.

### 3.2.1 Existing System:

### Naïve Bayes (NB):

In statistics, **naive Bayes classifiers** are a family of simple "probabilistic classifiers" based on applying Bayes' theorem with strong (naïve) independence assumptions between the features. They are among the simplest Bayesian network models, but coupled with kernel density estimation, they can achieve higher accuracy levels.

Naïve Bayes classifiers are highly scalable, requiring a number of parameters linear in the number of variables (features/predictors) in a learning problem. Maximum-likelihood training can be done by evaluating a closed-form expression, which takes linear time, rather than by expensive iterative approximation as used for many other types of classifiers.

In the statistics and computer science literature, naive Bayes models are known under a variety of names, including **simple Bayes** and **independence Bayes**. All these names reference the use of Bayes' theorem in the classifier's decision rule, but naïve Bayes is not (necessarily) a Bayesian method.

Naive Bayes is a simple technique for constructing classifiers: models that assign class labels to problem instances, represented as vectors of feature values, where the class labels are drawn from some finite set. There is not a single algorithm for training such classifiers, but a family of algorithms based on a common principle: all naive Bayes classifiers assume that the value of a particular feature is independent of the value of any other feature, given the class variable. For example, a fruit may be considered to be an apple if it is red, round, and about 10 cm in diameter. A naive Bayes classifier considers each of these features to contribute independently to the probability that this fruit is an apple, regardless of any possible correlations between the color, roundness, and diameter features.

For some types of probability models, naive Bayes classifiers can be trained very efficiently in a supervised learning setting. In many practical applications, parameter estimation for naive Bayes models uses the method of maximum likelihood; in other words, one can work with the naive Bayes model without accepting Bayesian probability or using any Bayesian methods.

Despite their naive design and apparently oversimplified assumptions, naive Bayes classifiers have worked quite well in many complex real-world situations. In 2004, an analysis of the Bayesian classification problem showed that there are sound theoretical reasons for the apparently implausible efficacy of naive Bayes classifiers. Still, a comprehensive comparison with other classification algorithms in 2006 showed that Bayes classification is outperformed by other approaches, such as boosted trees or random forests.

An advantage of naive Bayes is that it only requires a small number of training data to estimate the parameters necessary for classification.

### 3.2.2 Convolution Neural Network (CNN):

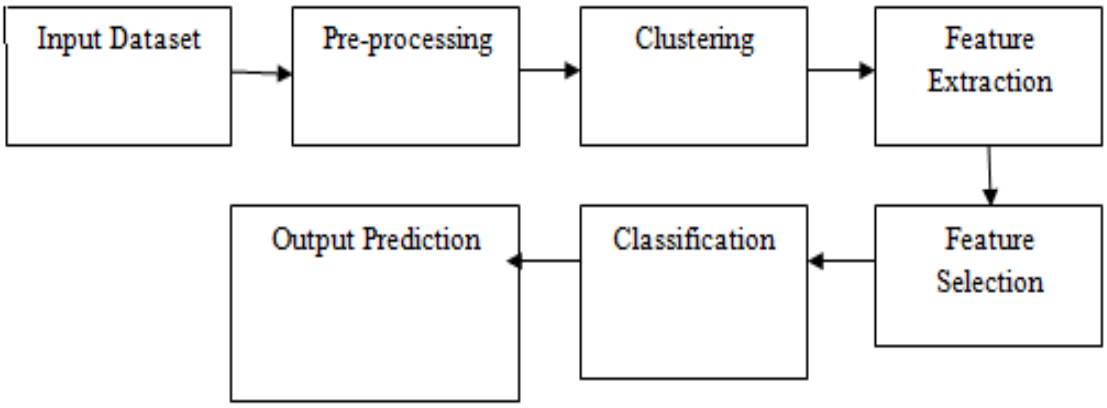
In deep learning, a **convolutional neural network** (**CNN**, or **ConvNet**) is a class of deep neural networks, most commonly applied to analyzing visual imagery. They are also known as **shift invariant** or **space invariant artificial neural networks** (**SIANN**), based on their shared-weights architecture and translation invariance characteristics. They have applications in image and video recognition, recommender systems, image classification, Image segmentation, medical image analysis, natural language processing, brain-computer interfaces, and financial time series.

CNNs are regularized versions of multilayer perceptions. Multilayer perceptions usually mean fully connected networks, that is, each neuron in one layer is connected to all neurons in the next layer. The "fully-connectedness" of these networks makes them prone to overfitting data. Typical ways of regularization include adding some form of magnitude measurement of weights to the loss function. CNNs take a different approach towards regularization: they take advantage of the hierarchical pattern in data and assemble more complex patterns using smaller and simpler patterns. Therefore, on the scale of connectedness and complexity, CNNs are on the lower extreme.

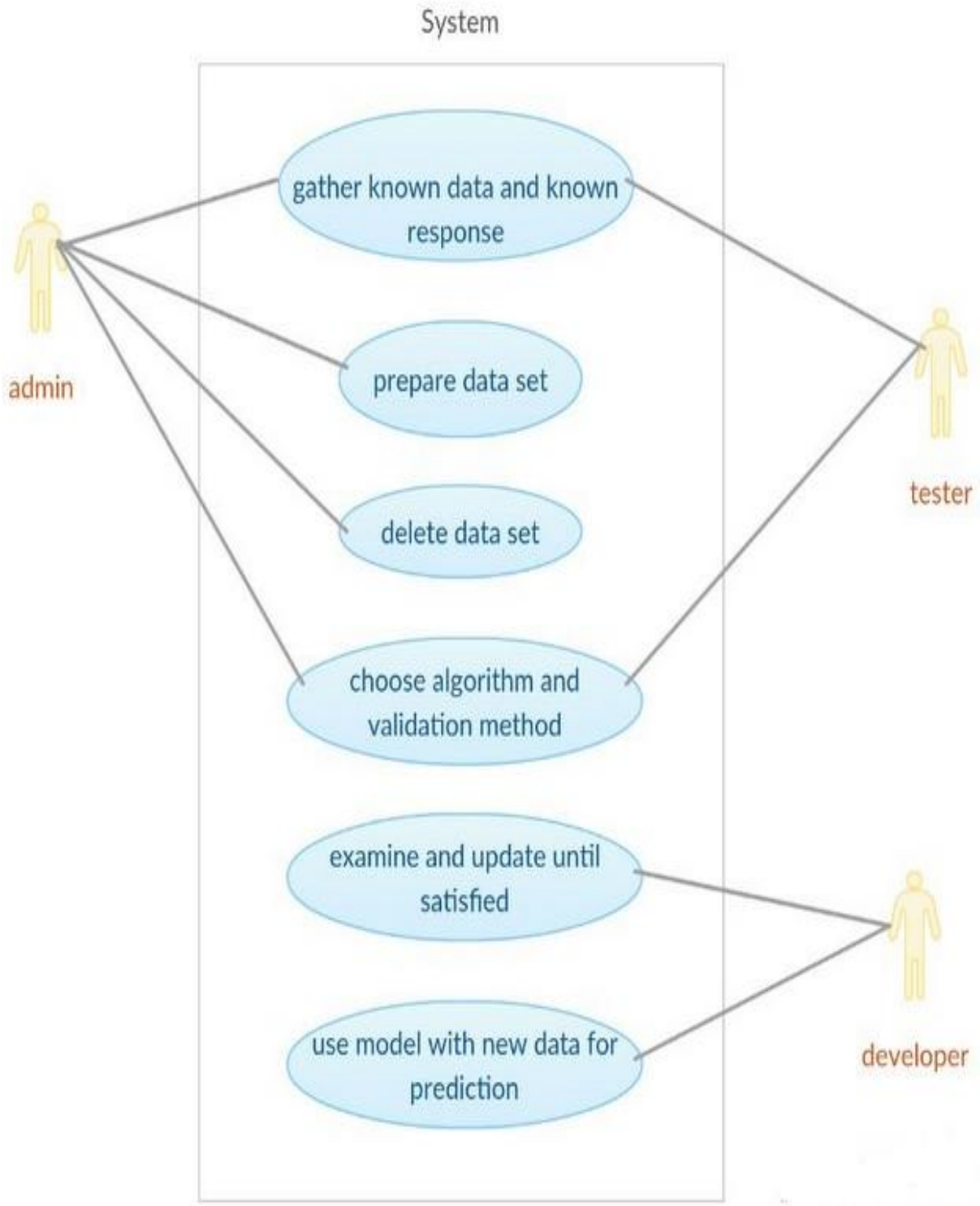
Convolutional networks were inspired by biological processes in that the connectivity pattern between neurons resembles the organization of the animal visual cortex. Individual cortical neurons respond to stimuli only in a restricted region of the visual field known as the receptive field. The receptive fields of different neurons partially overlap such that they cover the entire visual field.

CNNs use relatively little pre-processing compared to other image classification algorithms. This means that the network learns the filters that in traditional algorithms were hand-engineered. This independence from prior knowledge and human effort in feature design is a major advantage.

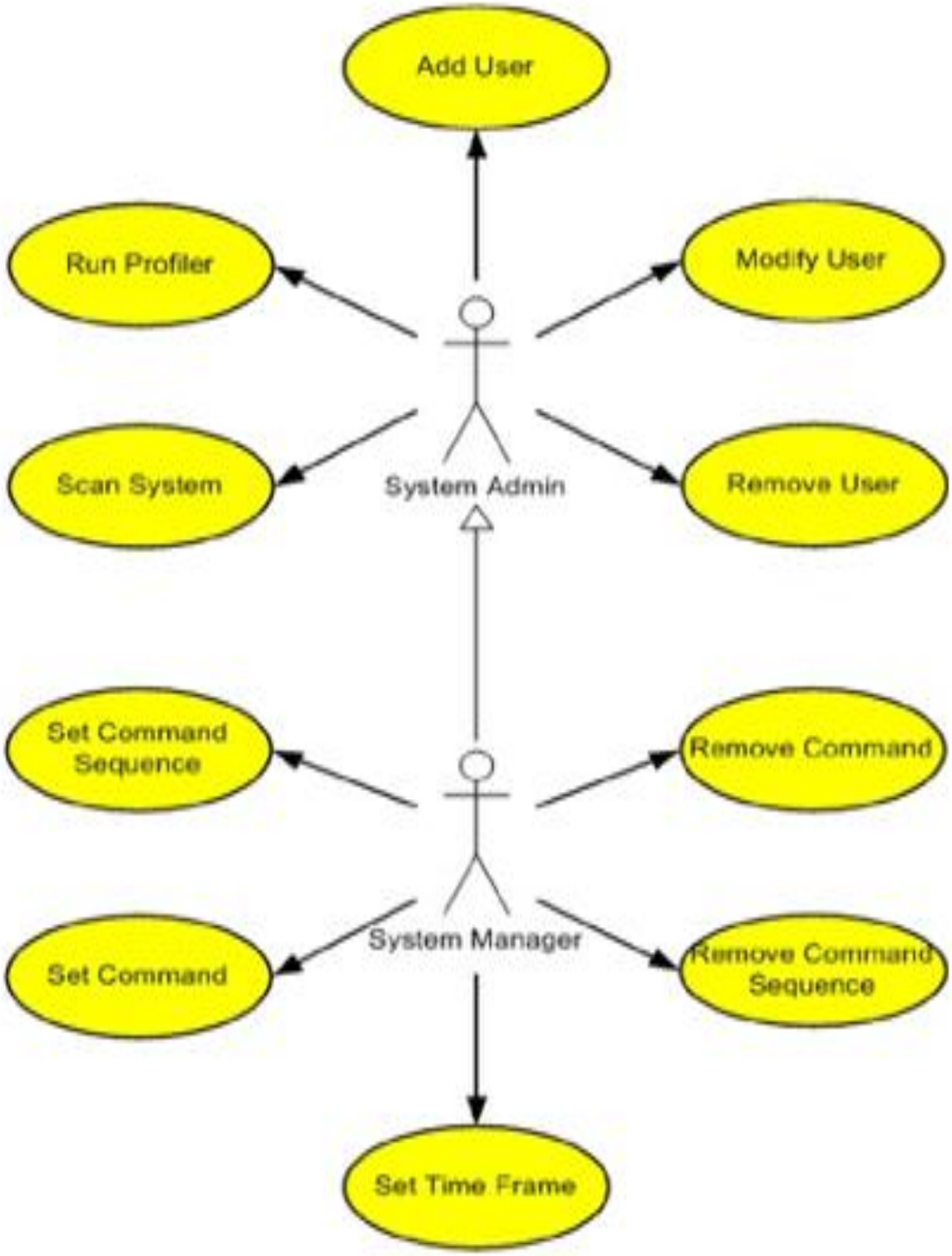
## 3.3 System Architecture:



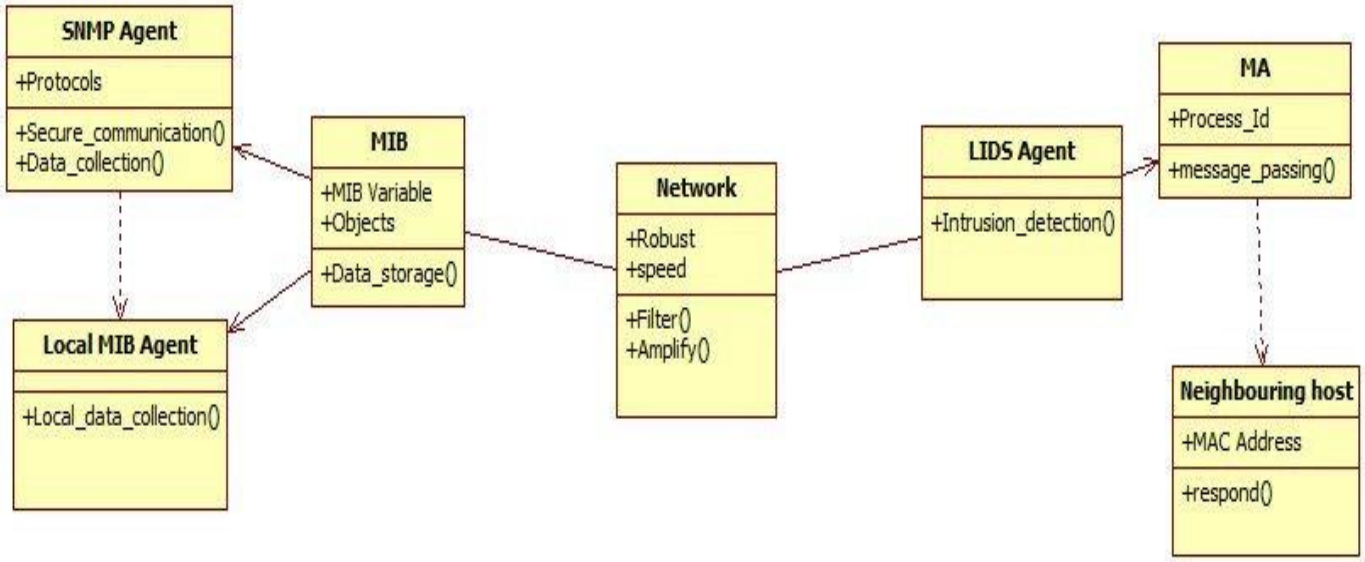
## 3.4 UML Diagram:



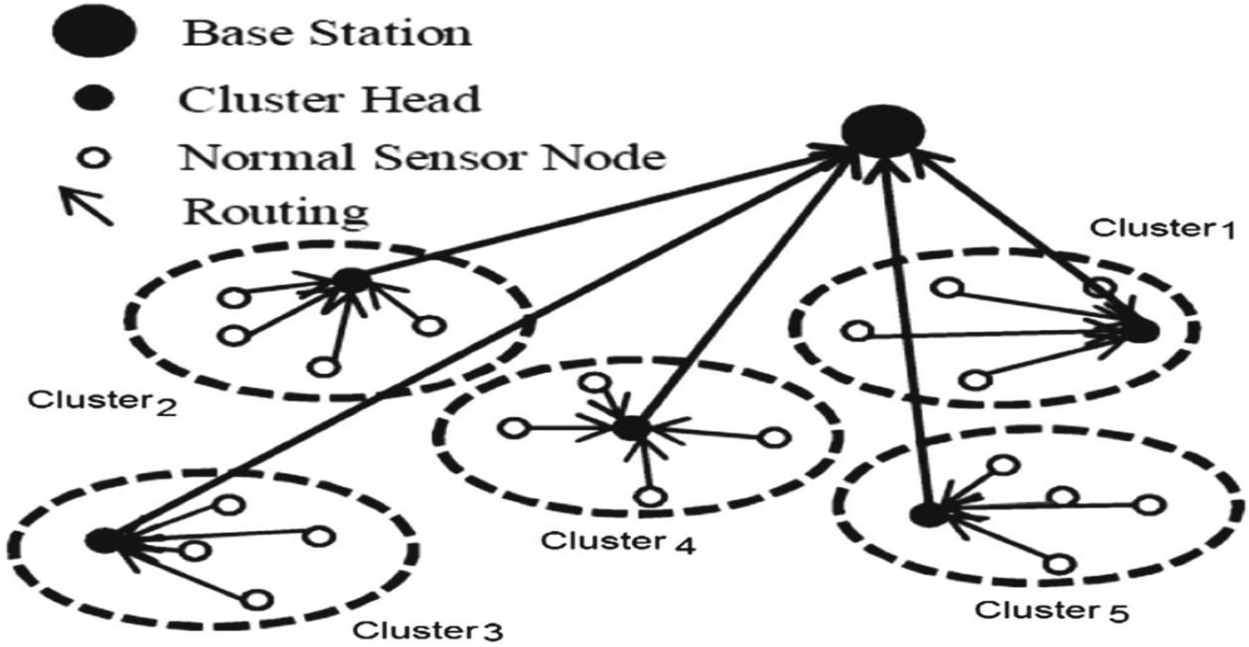
### 3.4.1 Use Case Diagram:



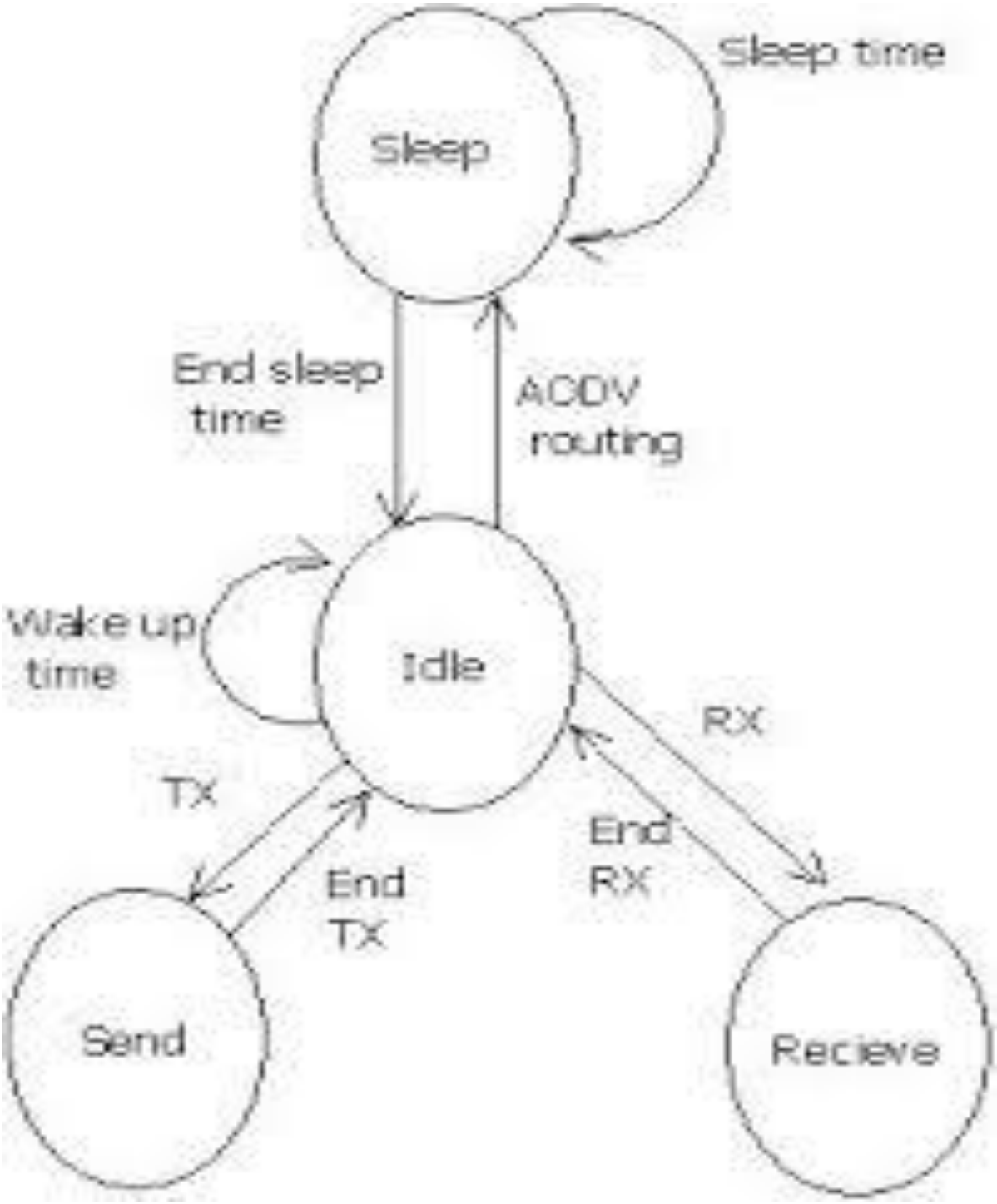
### 3.4.2 Class Diagram:




### 3.4.3 Component Diagram:



### 3.4.4 <u>Deployment Diagram:</u>



# CHAPTER 4

# 4. SYSTEM IMPLEMENTATION

## 4.1 Module Description:

There are 8 components in the system. They are,

- Incoming packet
- Packet Capture
- Packet scanner
- Packet analyzer
- Labeling
- Training model
- Prediction:
- Output

**Incoming packet:** In our project directly connected to network (we capture online packet) and transfer to packet scanner.

**Packet Capture:** In Real time networking many packets are transferred from source to destination. In this module live packets from the network are captured and passed to the pre-processor module. According to the concept of machine learning the data is grouped or clustered based on its features or characteristics for e.g. protocols used by the packets.

**Packet scanner:** After catching the packet we use packet scanner for the purpose of scanning the packet. Packet scanning is the important part of our system.

**Packet analyzer:** Packet analyzer is also known as Packet sniffer. As data streams flow across the network sniffer capture each packet and if needed decodes the packet, raw data showing the values of various fields in the packet and analyzes its content according to specification.

**Labeling:** Labeling is used for defining the corresponding packet.

**Training model:** Training model contain set of trained data set which used to detect attack in the packet.

**Prediction:** Based on the training model we make the prediction that the packet is normal packet or abnormal packet.

**Output:** Based on the prediction (i.e. normal or abnormal) we generate output in the form for abnormal packet

## 4.2 Algorithm of Proposed Work:

i. Dataset is divided into small overlapping or either non-overlapping blocks.

ii. Extract the features using traditional techniques.

iii. Extracted feature values corresponding to each key point are stored in matrix.

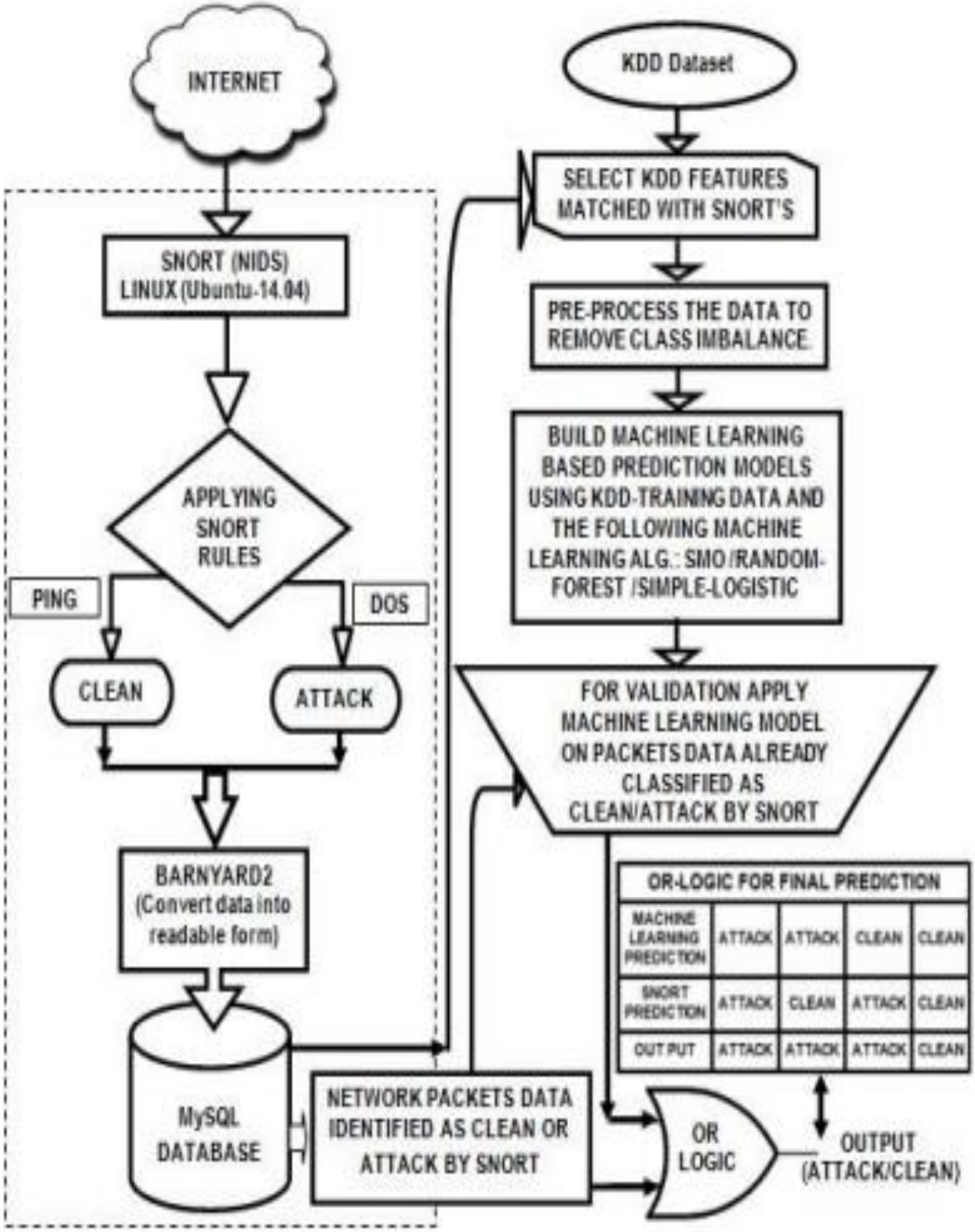
iv. Apply sorting techniques to get similar features that lie in nearness.

v. Introduce shift vector concept to find key point with similar shifting.

vi. Use the counter vector to count the occurrence same shifting key point and set the counter to 1.

vii. Similar regions are identified with the help of threshold value above steps are used.

## 4.3 Flow Chart:



## 4.4 Comparative Study of Existing and Proposed system:

In our project we have used algorithms like Naïve Bayes (NB) and Convolution Neural Network (CNN). All are measured in terms of accuracy.

From this we are getting good accuracy so we can state that our proposed system works better than the existing system.

# CHAPTER 5

# 5. SYSTEM STUDY

## 5.1 Feasibility Study:

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

The feasibility study investigates the problem and the information needs of the stakeholders. It seeks to determine the resources required to provide an information systems solution, the cost and benefits of such a solution, and the feasibility of such a solution. The analyst conducting the study gathers information using a variety of methods, the most popular of which are:

- Developing and administering questionnaires to interested stakeholders, such as potential users of the information system.
- Observing or monitoring users of the current system to determine their needs as well as their satisfaction and dissatisfaction with the current system.

- Collecting, examining, and analyzing documents, reports, layouts, procedures, manuals, and any other documentation relating to the operations of the current system.
- Modeling, observing, and simulating the work activities of the current system.

The goal of the feasibility study is to consider alternative information systems solutions, evaluate their feasibility, and propose the alternative most suitable to the organization.

The feasibility of a proposed solution is evaluated in terms of its components. These components are:

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY
- OPERATIONAL FEASIBILITY

## 5.2 Economical Feasibility:

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified.

Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

## 5.3 Technical Feasibility:

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources.

This will lead to high demands being placed on the client. The developed system must have a modest requirement as only minimal or null changes are required for implementing this system.

## 5.4 Social Feasibility:

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity.

The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

## 5.5 Operational Feasibility:

The ability, desire, and willingness of the stakeholders to use, support, and operate the proposed computer information system. The stakeholders include management, employees, customers, and suppliers.

The stakeholders are interested in systems that are easy to operate, make few, if any, errors, produce the desired information, and fall within the objectives of the organization.

# CHAPTER 6

# 6. SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product.

It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner.

There are various types of test. Each test type addresses a specific testing requirement.

## Types of Tests:

## 6.1 Unit testing:

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration.

This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration.

Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

## 6.2 Integration testing:

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields.

Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components

## 6.3 Functional testing:

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

## 6.4 System Testing:

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test.

System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

## 6.5 White Box Testing:

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

## 6.6 Black Box Testing:

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document.

It is a testing in which the software under test is treated, as a black box you cannot "see" into it. The test provides inputs and responds to outputs without considering how the software works.

## 6.7 Unit Testing:

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

## Test strategy and approach:

Field testing will be performed manually and functional tests will be written in detail.

## Test objectives:

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

## Features to be tested:

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

## 6.8 Integration Testing:

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects. The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

## Test Results:

All the test cases mentioned above passed successfully. No defects encountered.

## 6.9 Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

## Test Results:

All the test cases mentioned above passed successfully. No defects encountered.

# CHAPTER 7

# 7. SOFTWARE DESCRIPTION

## A. Python:

Python is a remarkably powerful dynamic, object-oriented programming language that is used in a wide variety of application domains. It offers strong support for integration with other languages and tools, and comes with extensive standard libraries. To be precise, the following are some distinguishing features of Python:

- Very clear, readable syntax.
- Strong introspection capabilities.
- Full modularity.
- Exception-based error handling.
- High level dynamic data types.
- Supports object oriented, imperative and functional programming styles.
- Embeddable.
- Scalable
- Mature

With so much of freedom, Python helps the user to think problem centric rather than language centric as in other cases. These features make Python a best option for scientific computing.

## B. Open CV:

Open CV is a library of programming functions for real time computer vision originally developed by Intel and now supported by Willogarage. It is free for use under the open source

BSD license. The library has more than five hundred optimized algorithms. It is used around the world, with forty thousand people in the user group. Uses range from interactive art, to mine inspection, and advanced robotics. The library is mainly written in C, which makes it portable to some specific platforms such as Digital Signal Processor.

Wrappers for languages such as C, Python, Ruby and Java (using Java CV) have been developed to encourage adoption by a wider audience. The recent releases have interfaces for C++. It focuses mainly on real-time image processing. Open CV is a cross-platform library, which can run on Linux, Mac OS and Windows. To date, Open CV is the best open source computer vision library that developers and researchers can think of.

### C. Tesseract:

Tesseract is a free software OCR engine that was developed at HP between 1984 and 1994. HP released it to the community in 2005. Tesseract was introduced at the 1995 UNLV Annual Test OCR Accuracy and is currently developed by Google released under the Apache License. It can now recognize 6 languages, and is fully UTF8 capable. Developers can train Tesseract with their own fonts and character mapping to obtain perfect efficiency.

## 7.1 About the software "PYTHON":

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted** − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive** − You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented** − Python supports Object-Oriented style or technique of programming that encapsulates code within objects.

- **Python is a Beginner's Language** − Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

## History of Python:

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, Small Talk, and Unix shell and other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

## 7.2 Python Features:

Python's features include −

- **Easy-to-learn** − Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read** − Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain** − Python's source code is fairly easy-to maintain
- **A broad standard library** − Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode** − Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.

- **Portable** − Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable** − You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases** − Python provides interfaces to all major commercial databases.
- **GUI Programming** − Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable** − Python provides a better structure and support for large programs than shell scripting.

Apart from the above-mentioned features, Python has a big list of good features, few are listed below −

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- It supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.

## 7.3 Python - Environment Setup:

Python is available on a wide variety of platforms including Linux and Mac OS X. Let's understand how to set up our Python environment.

## Local Environment Setup:

Open a terminal window and type "python" to find out if it is already installed and which version is installed.

- Unix (Solaris, Linux, FreeBSD, AIX, HP/UX, SunOS, IRIX, etc.)
- Win 9x/NT/2000
- Macintosh (Intel, PPC, 68K)
- OS/2
- DOS (multiple versions)
- PalmOS
- Nokia mobile phones
- Windows CE
- Acorn/RISC OS
- BeOS
- Amiga
- VMS/OpenVMS
- QNX
- VxWorks
- Psion
- Python has also been ported to the Java and .NET virtual machines

## 7.4 Getting Python:

The most up-to-date and current source code, binaries, documentation, news, etc., is available on the official website of Python https://www.python.org/

You can download Python documentation https://www.python.org/doc/The documentation is available in HTML, PDF, and PostScript formats.

## 7.5 Installing Python:

Python distribution is available for a wide variety of platforms. You need to download only the binary code applicable for your platform and install Python.

If the binary code for your platform is not available, you need a C compiler to compile the source code manually. Compiling the source code offers more flexibility in terms of choice of features that you require in your installation.

Here is a quick overview of installing Python on various platforms −

Unix and Linux Installation

Here are the simple steps to install Python on Unix/Linux machine.

- Open a Web browser and go to https://www.python.org/downloads/.
- Follow the link to download zipped source code available for Unix/Linux.
- Download and extract files.
- Editing the *Modules/Setup* file if you want to customize some options.
- run ./configure script
- make
- make install

This installs Python at standard location */usr/local/bin* and its libraries at */usr/local/lib/pythonXX* where XX is the version of Python.

Windows Installation

Here are the steps to install Python on Windows machine.

- Open a Web browser and go to https://www.python.org/downloads/.
- Follow the link for the Windows installer *python-XYZ.msi* file where XYZ is the version you need to install.

- To use this installer *python-XYZ.msi*, the Windows system must support Microsoft Installer 2.0. Save the installer file to your local machine and then run it to find out if your machine supports MSI.
- Run the downloaded file. This brings up the Python install wizard, which is really easy to use. Just accept the default settings, wait until the install is finished, and you are done.

## 7.6 Macintosh Installation:

Recent Macs come with Python installed, but it may be several years out of date. See http://www.python.org/download/mac/ for instructions on getting the current version along with extra tools to support development on the Mac. For older Mac OS's before Mac OS X 10.3 (released in 2003), MacPython is available.

Jack Jansen maintains it and you can have full access to the entire documentation at his website − http://www.cwi.nl/~jack/macpython.html. You can find complete installation details for Mac OS installation.

## 7.7 Setting up PATH:

Programs and other executable files can be in many directories, so operating systems provide a search path that lists the directories that the OS searches for executables.

The path is stored in an environment variable, which is a named string maintained by the operating system. This variable contains information available to the command shell and other programs.

The **path** variable is named as PATH in Unix or Path in Windows (Unix is case sensitive; Windows is not).

In Mac OS, the installer handles the path details. To invoke the Python interpreter from any particular directory, you must add the Python directory to your path.

## Setting path at Unix/Linux:

To add the Python directory to the path for a particular session in Unix −

- **In the csh shell** − type setenv PATH "$PATH:/usr/local/bin/python" and press Enter.
- **In the bash shell (Linux)** − type export PATH="$PATH:/usr/local/bin/python" and press Enter.
- **In the sh or ksh shell** − type PATH="$PATH:/usr/local/bin/python" and press Enter.
- **Note** − /usr/local/bin/python is the path of the Python directory

## Setting path at Windows:

To add the Python directory to the path for a particular session in Windows −

**At the command prompt** − type path %path%;C:\Python and press Enter.

**Note** − C:\Python is the path of the Python directory

## Python Environment Variables:

Here are important environment variables, which can be recognized by Python −

| Sr.No. | Variable & Description |
|---|---|
| 1 | **PYTHONPATH**<br><br>It has a role similar to PATH. This variable tells the Python interpreter where to locate the module files imported into a program. It should include the Python source library directory and the directories containing Python source code. PYTHONPATH is sometimes preset by the Python installer. |

| | |
|---|---|
| 2 | **PYTHONSTARTUP**<br><br>It contains the path of an initialization file containing Python source code. It is executed every time you start the interpreter. It is named as .pythonrc.py in Unix and it contains commands that load utilities or modify PYTHONPATH. |
| 3 | **PYTHONCASEOK**<br><br>It is used in Windows to instruct Python to find the first case-insensitive match in an import statement. Set this variable to any value to activate it. |
| 4 | **PYTHONHOME**<br><br>It is an alternative module search path. It is usually embedded in the PYTHONSTARTUP or PYTHONPATH directories to make switching module libraries easy. |

## 7.8 Running Python:

There are three different ways to start Python −

Interactive Interpreter

You can start Python from Unix, DOS, or any other system that provides you a command-line interpreter or shell window.

Enter **python** the command line.

Start coding right away in the interactive interpreter.

```
$python # Unix/Linux
or
python% # Unix/Linux
```

```
or
C:> python # Windows/DOS
```

Here is the list of all the available command line options −

| Sr.No. | Option & Description |
|---|---|
| 1 | **-d**<br><br>It provides debug output. |
| 2 | **-O**<br><br>It generates optimized bytecode (resulting in .pyo files). |
| 3 | **-S**<br><br>Do not run import site to look for Python paths on startup. |
| 4 | **-v**<br><br>verbose output (detailed trace on import statements). |
| 5 | **-X**<br><br>disable class-based built-in exceptions (just use strings); obsolete starting with version 1.6. |
| 6 | **-c cmd**<br><br>run Python script sent in as cmd string |

| | |
|---|---|
| 7 | **File**<br><br>run Python script from given file |

Script from the Command-line

A Python script can be executed at command line by invoking the interpreter on your application, as in the following –

```
$python script.py # Unix/Linux

or

python% script.py # Unix/Linux

or

C: >python script.py # Windows/DOS
```

**Note** − Be sure the file permission mode allows execution.

Integrated Development Environment

You can run Python from a Graphical User Interface (GUI) environment as well, if you have a GUI application on your system that supports Python.

- **Unix** − IDLE is the very first Unix IDE for Python.
- **Windows** − PythonWin is the first Windows interface for Python and is an IDE with a GUI.
- **Macintosh** − The Macintosh version of Python along with the IDLE IDE is available from the main website, downloadable as either MacBinary or BinHex'd files.

If you are not able to set up the environment properly, then you can take help from your system admin. Make sure the Python environment is properly set up and working perfectly fine.

**Note** − All the examples given in subsequent chapters are executed with Python 2.4.3 version available on CentOS flavor of Linux.

We already have set up Python Programming environment online, so that you can execute all the available examples online at the same time when you are learning theory. Feel free to modify any example and execute it online.

## 7.9 Panda:

Pandas is an open-source, BSD-licensed Python library providing high-performance, easy-to-use data structures and data analysis tools for the Python programming language. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc. In this tutorial, we will learn the various features of Python Pandas and how to use them in practice.

## Audience:

This tutorial has been prepared for those who seek to learn the basics and various functions of Pandas. It will be specifically useful for people working with data cleansing and analysis. After completing this tutorial, you will find yourself at a moderate level of expertise from where you can take yourself to higher levels of expertise.

## Prerequisites:

You should have a basic understanding of Computer Programming terminologies. A basic understanding of any of the programming languages is a plus. Pandas library uses most of the functionalities of NumPy. It is suggested that you go through our tutorial on NumPy before proceeding with this tutorial. You can access it from Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data

structures. The name Pandas is derived from the word Panel Data – an Econometrics from Multidimensional data.

In 2008, developer Wes McKinney started developing pandas when in need of high performance, flexible tool for analysis of data.

Prior to Pandas, Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data — load, prepare, manipulate, model, and analyze.

Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

## 7.10 Key Features of Pandas:

- Fast and efficient DataFrame object with default and customized indexing.
- Tools for loading data into in-memory data objects from different file formats.
- Data alignment and integrated handling of missing data.
- Reshaping and pivoting of date sets.
- Label-based slicing, indexing and subsetting of large data sets.
- Columns from a data structure can be deleted or inserted.
- Group by data for aggregation and transformations.
- High performance merging and joining of data.
- Time Series functionality.

Standard Python distribution doesn't come bundled with Pandas module.

## NumPy:

NumPy, which stands for Numerical Python, is a library consisting of multidimensional array objects and a collection of routines for processing those arrays. Using NumPy, mathematical and logical operations on arrays can be performed. This tutorial explains the basics of NumPy such as its architecture and environment. It also discusses the various array functions, types of

indexing, etc. An introduction to Matplotlib is also provided. All this is explained with the help of examples for better understanding.

## Audience:

This tutorial has been prepared for those who want to learn about the basics and various functions of NumPy. It is specifically useful for algorithm developers. After completing this tutorial, you will find yourself at a moderate level of expertise from where you can take yourself to higher levels of expertise.

## Prerequisites:

You should have a basic understanding of computer programming terminologies. A basic understanding of Python and any of the programming languages is a plus.umPy is a Python package. It stands for 'Numerical Python'. It is a library consisting of multidimensional array objects and a collection of routines for processing of array.

## Numeric:

The ancestor of NumPy, was developed by Jim Hugunin. Another package Numarray was also developed, having some additional functionalities. In 2005, Travis Oliphant created NumPy package by incorporating the features of Numarray into Numeric package. There are many contributors to this open source project.

Operations using NumPy

Using NumPy, a developer can perform the following operations −

- Mathematical and logical operations on arrays.
- Fourier transforms and routines for shape manipulation.
- Operations related to linear algebra. NumPy has in-built functions for linear algebra and random number generation.

## NumPy – A Replacement for MatLab:

NumPy is often used along with packages like **SciPy** (Scientific Python) and **Mat−plotlib** (plotting library). This combination is widely used as a replacement for MatLab, a popular platform for technical computing. However, Python alternative to MatLab is now seen as a more modern and complete programming language.

It is open source, which is an added advantage of NumPy.tandard Python distribution doesn't come bundled with NumPy module. A lightweight alternative is to install NumPy using popular Python package installer, **pip**.

```
pip install numpy
```

The best way to enable NumPy is to use an installable binary package specific to your operating system. These binaries contain full SciPy stack (inclusive of NumPy, SciPy, matplotlib, IPython, SymPy and nose packages along with core Python).

# CHAPTER 8

# 8. CONCLUSION AND FUTURE SCOPE

## 8.1 Conclusion:

Intrusion detection is currently attracting interest from both the research community and commercial companies. We have given background of the current state-of-the-art of IDS, based on a proposed taxonomy illustrated with examples of past and current projects.

This taxonomy also highlights the recent work and covers the past and current developments adequately. Each of its technique has its own advantages and disadvantages.

We believe that no single criterion can be used to completely defend against computer network intrusion. There is no single version of it that can be used as a standard solution against all possible attacks. It is both technically difficult and economically costly to build and maintain computer systems and networks that are not susceptible to attacks.

The technique to be selected depends on the specifications of the type of anomalies that the system is supposed to face, the type and behavior of the data, the environment in which the system is working, the cost and computation limitations and the security level required.

## 8.2 Future Scope:

Future research should consider other machine learning algorithms to ascertain more efficient ways to perform the classification technique on the datasets.

It is recommended that further research should be carry out on other parameters that can improve the accuracy of detection.

# REFERENCES

[1] I. F. Akyildiz et al., “Wireless Sensor Networks: A Survey, “Elsevier Comp. Networks, vol. 3, no. 2, 2019, pp. 393–422

[2] G.Li, J.He, Y. Fu. “Group-based intrusion detection system in wireless sensor networks” Computer Communications, Volume 31, Issue 18 (December 2019)

[3] Michael Brownfield, “Wireless Sensor Network Denial of Sleep Attack”, Proceedings of the 2019 IEEE Workshop on Information Assurance and Security United States Military Academy, West Point, NY.

[4] FarooqAnjum, DhanantSubhadrabandhu, SaswatiSarkar *, Rahul Shetty, “On Optimal Placement of Intrusion Detection Modules in Sensor Networks”, Proceedings of the First International Conference on Broadband Networks (BROADNETS19).

[5] Parveen Sadotra et al, International Journal of Computer Science and Mobile Computing, Vol.5 Issue.9, September- 2019, pg. 23-28

[6] K. Akkayaand M. Younis, ―A Survey of Routing Protocols in Wireless Sensor Networks, ‖ in the Elsevier Ad Hoc Network Journal, Vol. 3/3 pp. 325-349, 2019.

[7] A. Abduvaliyev, S. Lee, Y.K Lee, “Energy Efficient Hybrid Intrusion Detection System for Wireless Sensor Networks”, IEEE International Conference on Electronics and Information Engineering, Vol.2, pp. 25-29, August 2019.

[8] Parveen Sadotra and Chandrakant Sharma. A Survey: Intelligent Intrusion Detection System in Computer Security. International Journal of Computer Applications 151(3):18-22, October 2019.

[9] A. Araujo, J. Blesa, E. Romero, D. Villanueva, “Security in cognitive wireless sensor networks. Challenges and open problems”, EURASIP Journal on Wireless Communications and Networking, February 2019.

[10] A. Becher, Z. Benenson, and M. Dorsey, \Tampering with motes: Real-world physical attacks on wireless sensor networks." in SPC (J. A. Clark, R. F. Paige, F. Polack, and P. J.Brooke, eds.), vol. 3934 of Lecture Notes in Computer Science, pp. 104{118, Springer, 2019.

[11] I. Krontiris and T. Dimitriou, \A practical authentication scheme for in-network programming in wireless sensor networks," in ACM Workshop on Real- World Wireless Sensor Networks, 2019.

[12] M. Ali Aydın *, A. HalimZaim, K. GokhanCeylan “A hybrid intrusion detection system design for computer network security” Computers and Electrical Engineering 35 (2019) 517–526.

# APPENDIX I

# OUTPUT SCREENSHOTS

## Sample Dataset:

Out[3]:

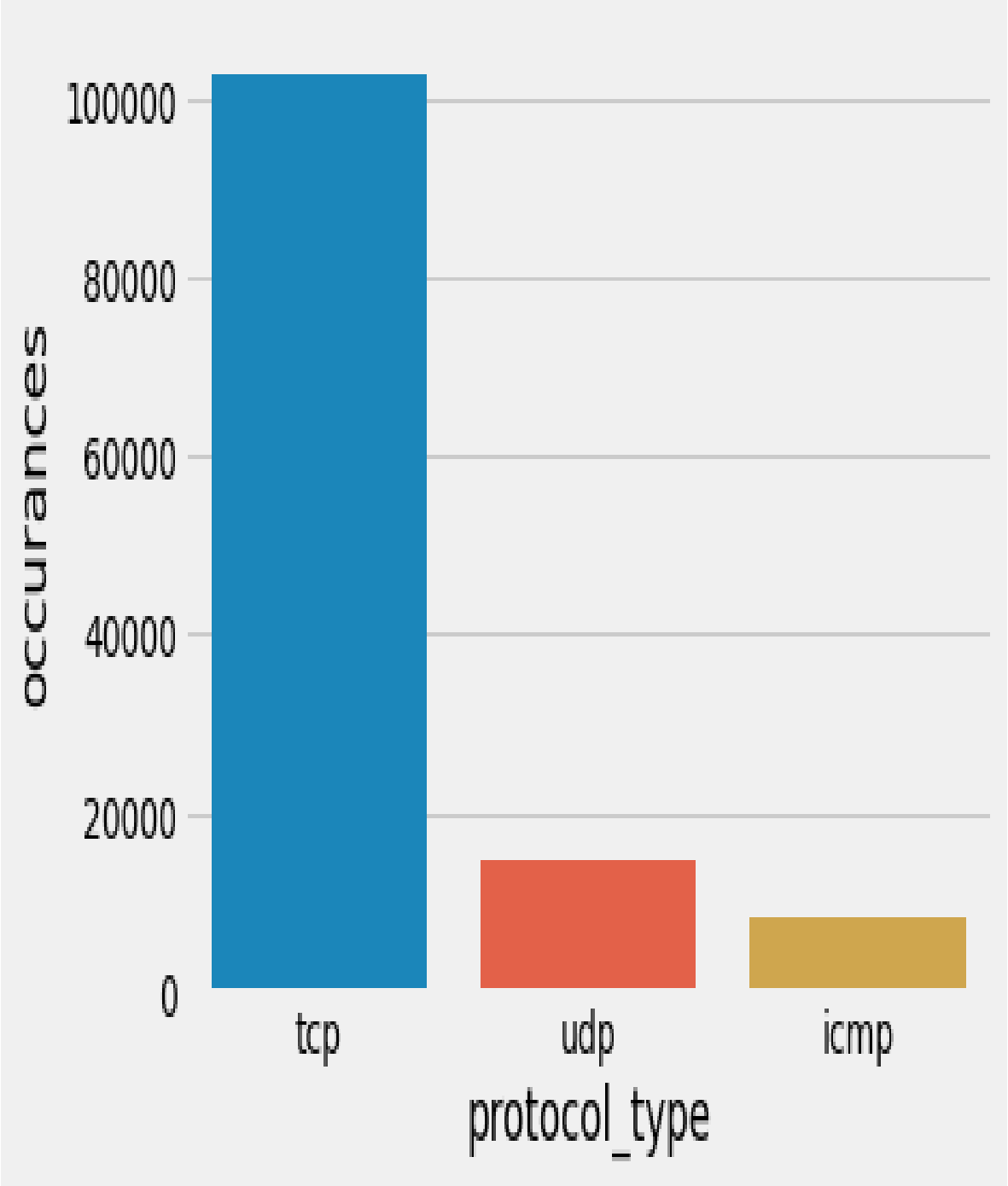
| | 0 | tcp | ftp_data | SF | 491 | 0.1 | 0.2 | 0.3 | 0.4 | 0.5 | 0.6 | 0.7 | 0.8 | 0.9 | 0.10 | 0.11 | 0.12 | 0.13 | 0.14 | 0.15 | 0.16 | 0.17 | 2 | 2.1 | 0.00 | 0.00.1 | 0.00.2 | 0.00.3 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | udp | other | SF | 146 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 13 | 1 | 0.0 | 0.0 | 0.0 | 0.0 | 0 |
| 1 | 0 | tcp | private | S0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 123 | 6 | 1.0 | 1.0 | 0.0 | 0.0 | 0 |
| 2 | 0 | tcp | http | SF | 232 | 8153 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 5 | 5 | 0.2 | 0.2 | 0.0 | 0.0 | 1 |
| 3 | 0 | tcp | http | SF | 199 | 420 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 30 | 32 | 0.0 | 0.0 | 0.0 | 0.0 | 1 |
| 4 | 0 | tcp | private | REJ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 121 | 19 | 0.0 | 0.0 | 1.0 | 1.0 | 0 |

## Outlier Removal:

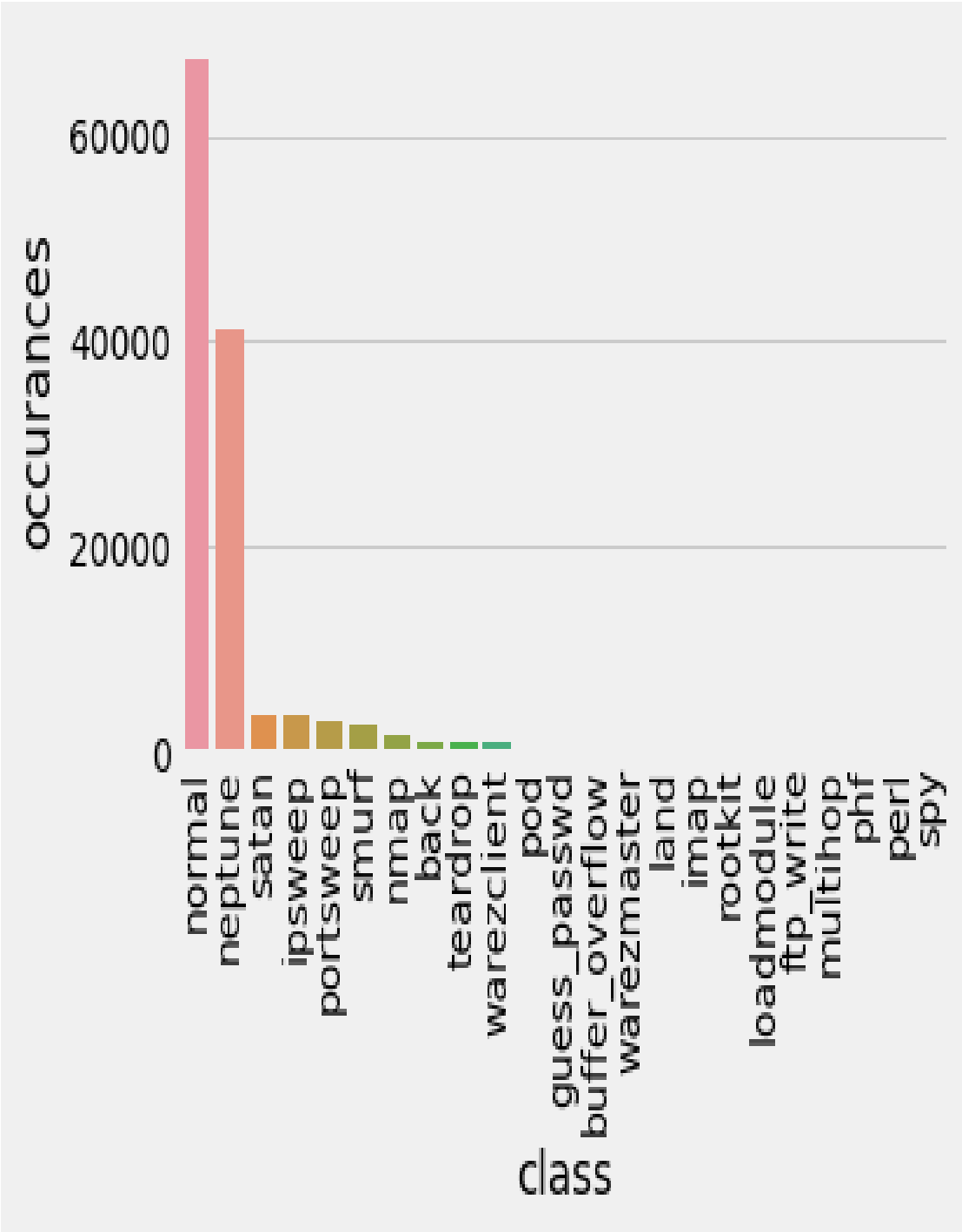
```
Z-score Calculation based on class to check outliers
```

```
Out[7]: array([-1.96554725, -0.21996766,  0.65282213, ..., -0.65636256,
               0.21642723,  0.65282213])
```

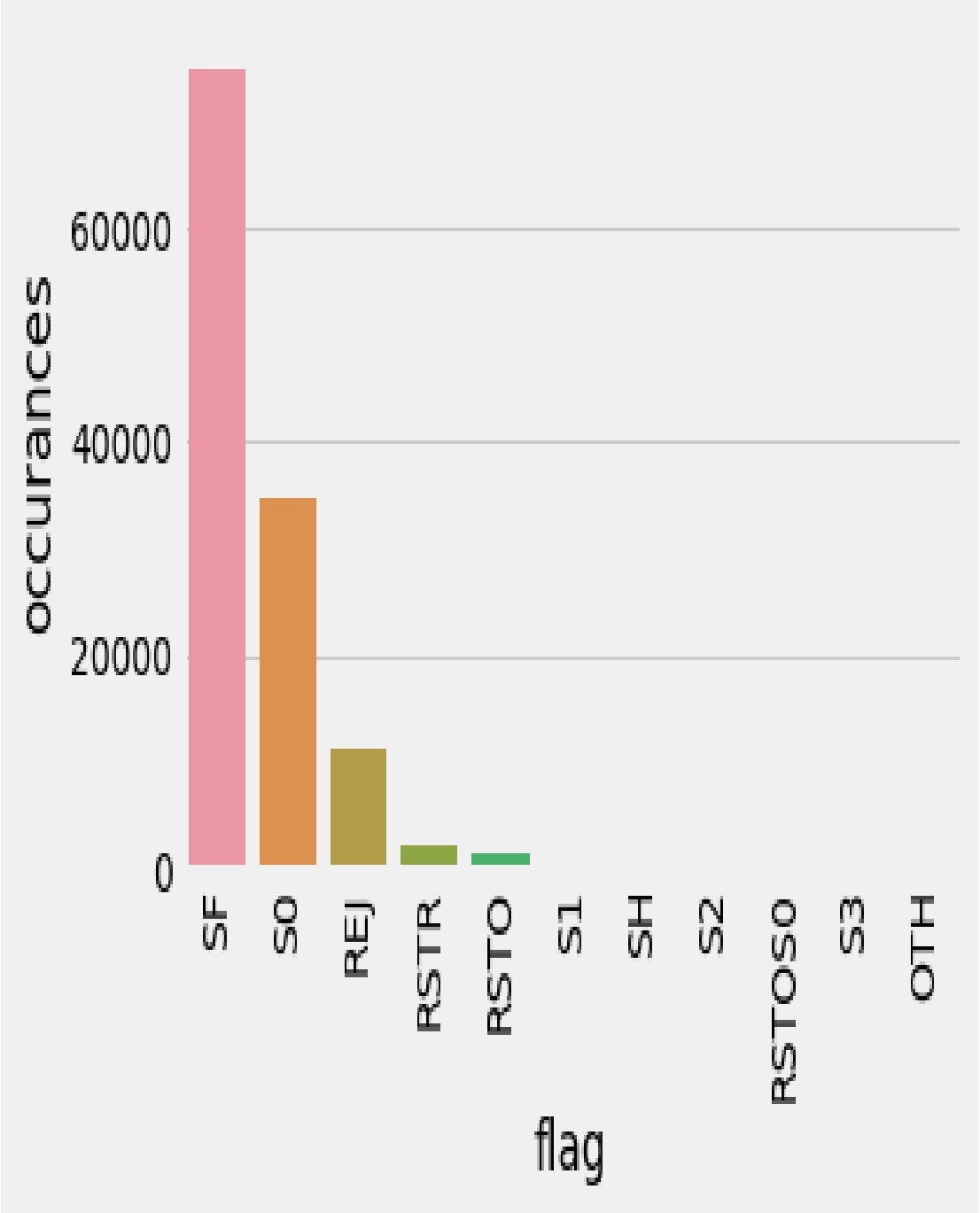
**Attack Based on Protocol:**



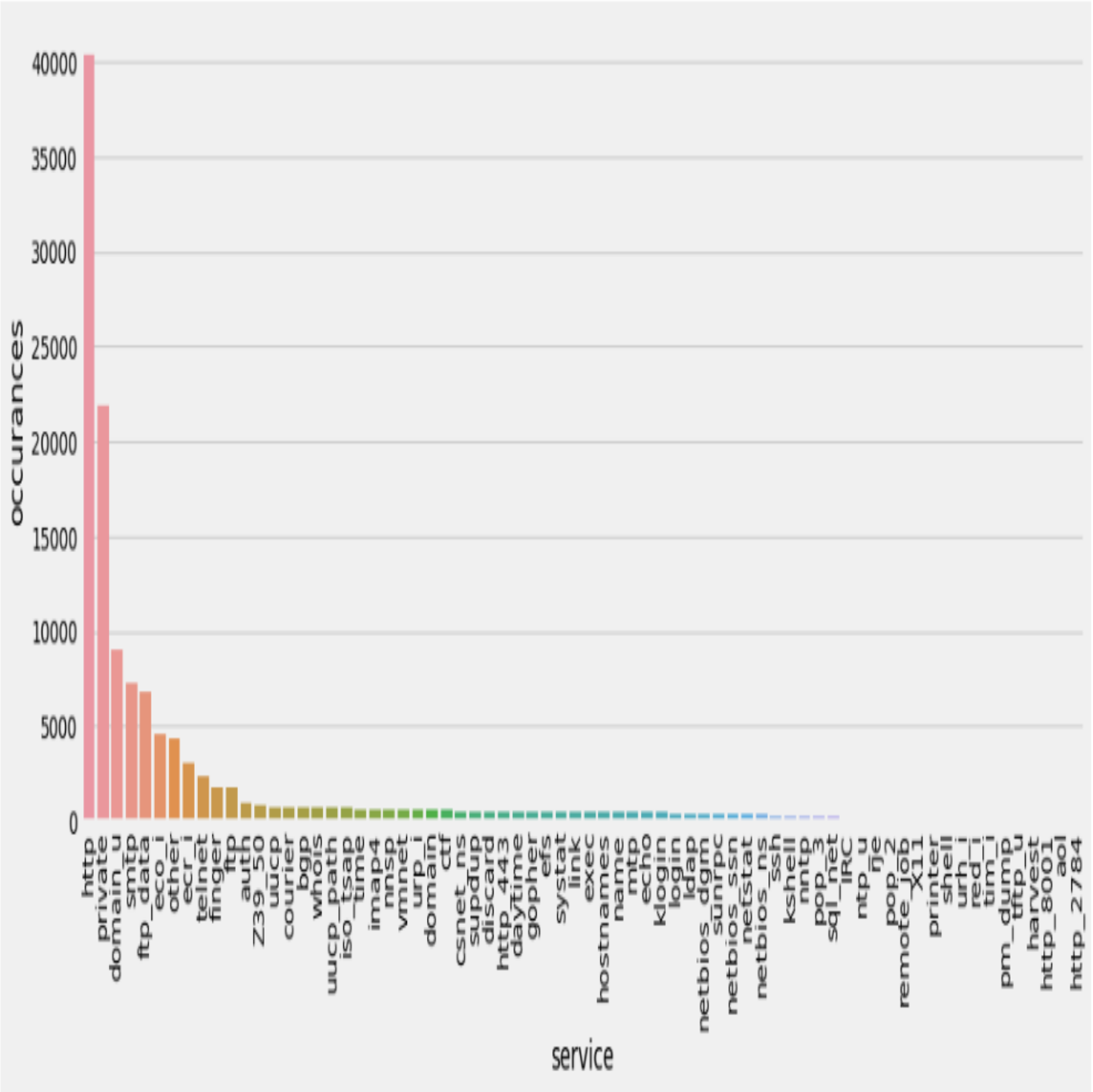
**Class Distribution:**



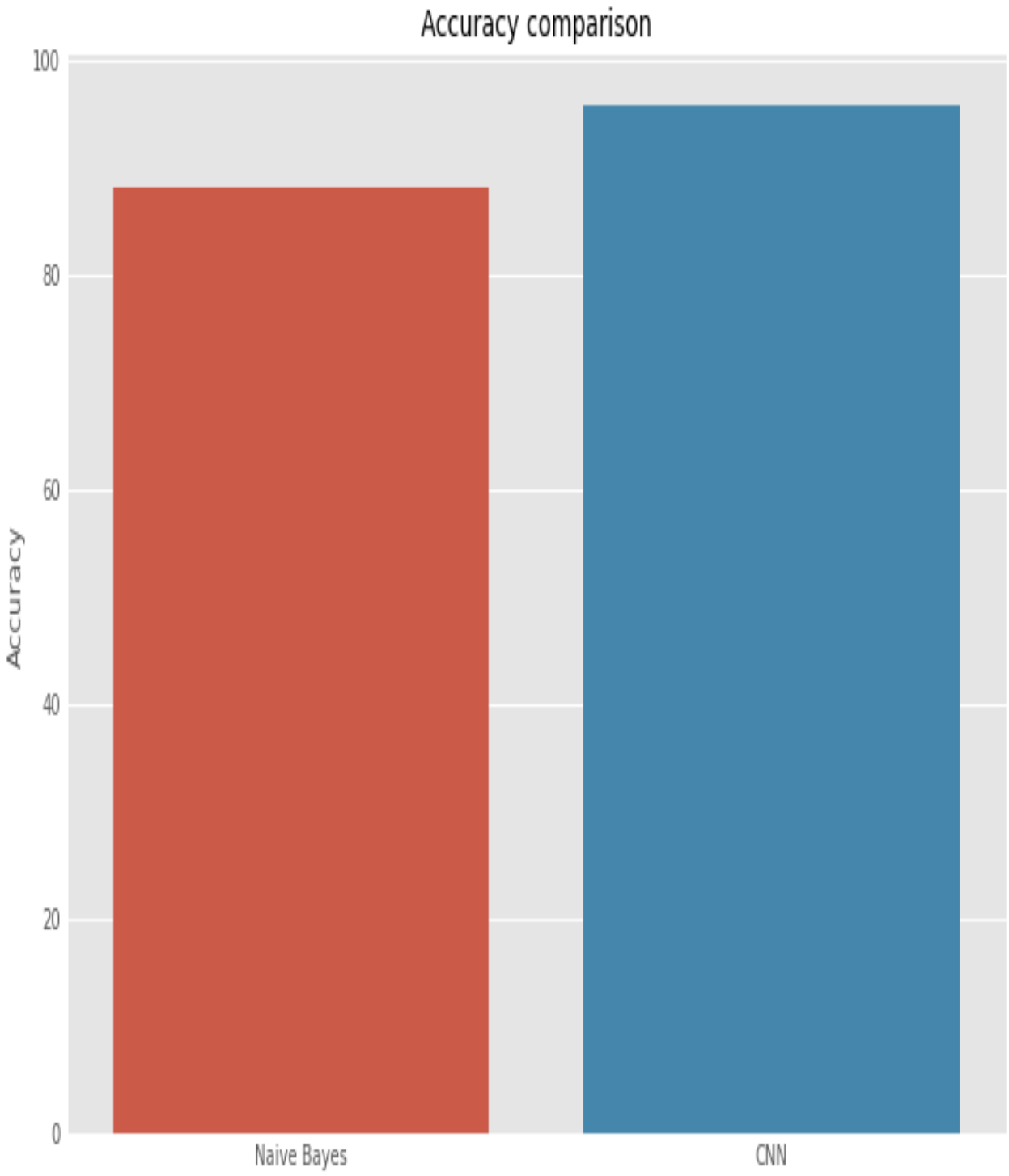
**Flag Count:**


occurances
60000
40000
20000
0
SF
SO
REJ
RSTR
RSTO
S1
SH
S2
RSTOSO
S3
OTH
flag

## Service Occurences:



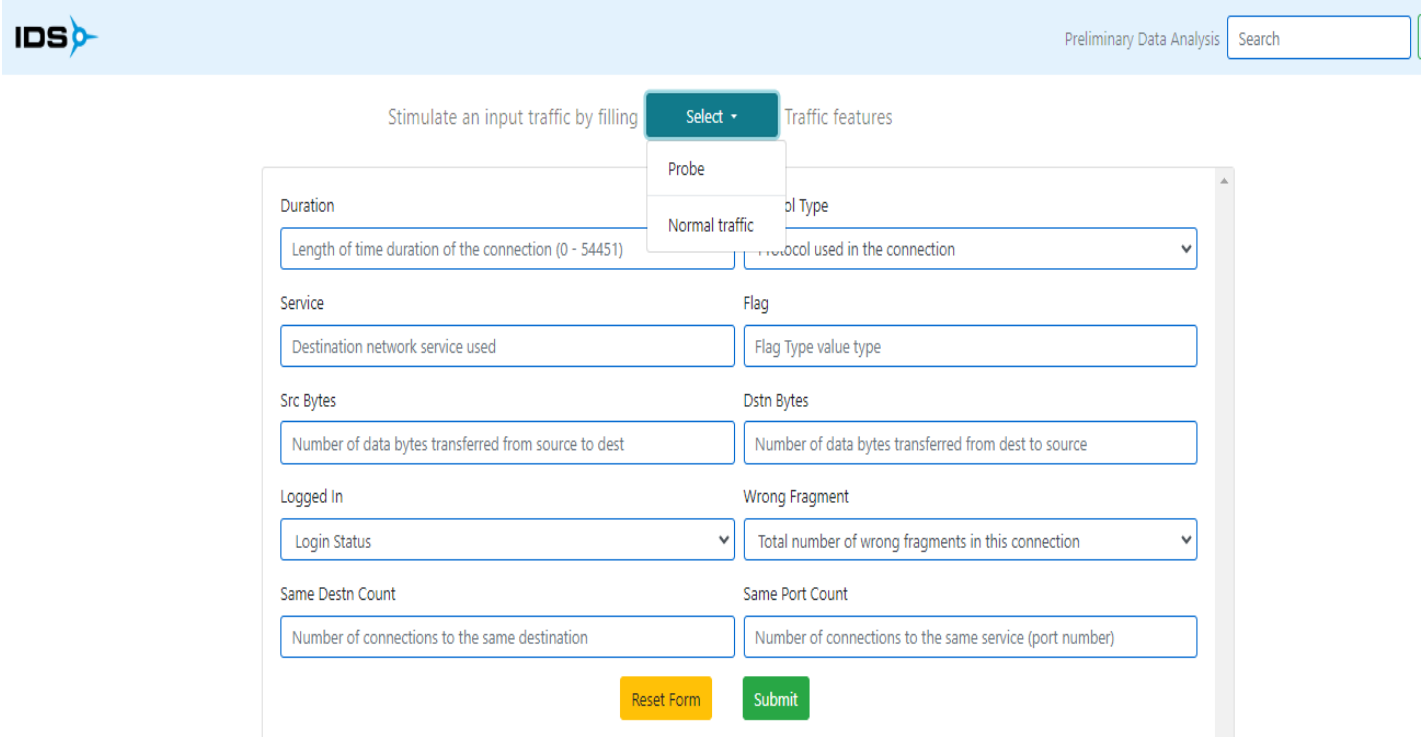
**Accuracy:**


Accuracy comparison
100
80
60
40
20
0
Accuracy
Naive Bayes
CNN

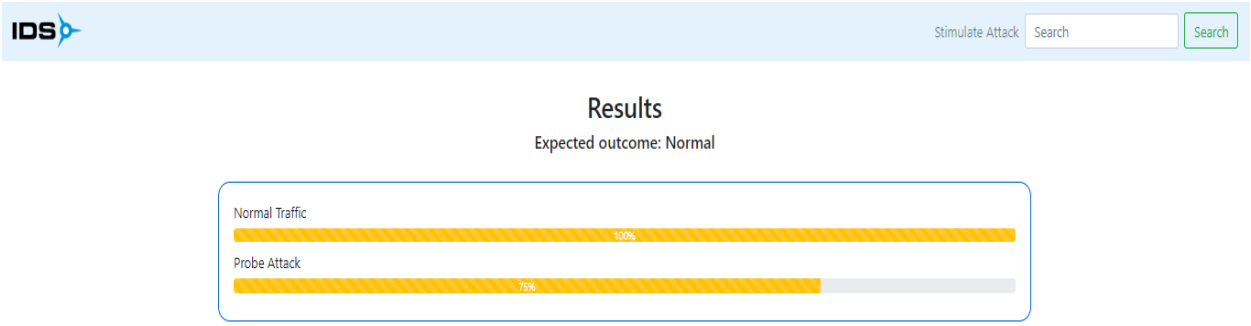
**Test set Prediction:**

```
No attack detected
No attack detected
No attack detected
Probe attack detected
No attack detected
No attack detected
No attack detected
No attack detected
No attack detected
No attack detected
Probe attack detected
No attack detected
Probe attack detected
Probe attack detected
No attack detected
Probe attack detected
No attack detected
Probe attack detected
Probe attack detected
```

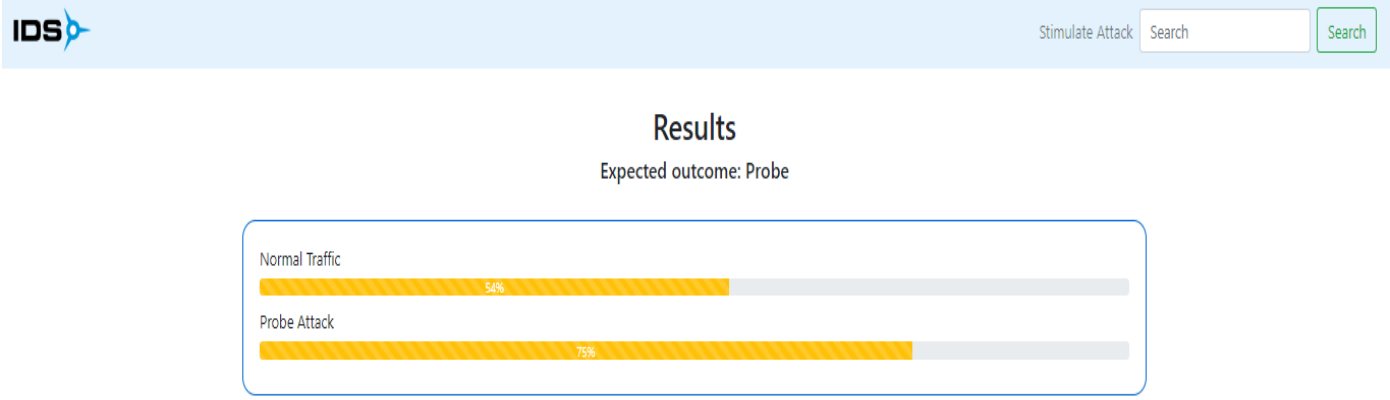
## Home Screen:




## Normal Prediction:



## Probe Detection:



# APPENDIX II

# PROGRAM CODING

```
<!DOCTYPE html>

<html lang="en">

<head>

  <title>IDS with ML</title>

  <meta charset="utf-8">

  <meta name="viewport" content="width=device-width, initial-scale=1">

  <link rel="shortcut icon" href="./static/images/favicon.png">

  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js"></script>

  <script
src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.16.0/umd/popper.min.js"></script
>

  <script
src="https://maxcdn.bootstrapcdn.com/bootstrap/4.4.1/js/bootstrap.min.js"></script>

  <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/4.4.1/css/bootstrap.min.css">

  <script src="./static/scripts/index.js"></script>



  <style>

    .form-control{
```

```
      border:1px solid #0068d0;
    }
  </style>
</head>
<body  style="overflow:hidden">


  <!-- Nav Bar -->
    <nav class="navbar navbar-expand-sm navbar-light fixed-top" style="background-color: #e3f2fd;">
      <!-- Brand/logo -->
      <ul class="navbar-nav mr-auto">
       <li><a class="navbar-brand" href="/"><img src="./static/images/logo.png" alt="Logo" style="height:40px;"></a></li>
      </ul>


      <!-- Links -->
      <ul class="navbar-nav mr-right">
        <li class="nav-item">
          <a class="nav-link" href="/analysis">Preliminary Data Analysis</a>
        </li>
```

```
        <form class="form-inline">
          <input class="form-control mr-sm-2" type="text" placeholder="Search"
id="Search">
        </form>
        <button class="btn btn-outline-success" onclick="find_text()">Search</button>
      </ul>
    </nav>


    <br><br><br>


    <!-- Header -->
    <nav class="navbar navbar-expand-sm navbar-light">


      <!-- Links -->
        <ul class="navbar-nav" style="margin-left: 25%;font-size: larger">
          <li class="nav-item">
            <a class="nav-link" href="#">Stimulate an input traffic by filling</a>
          </li>
          <form class="form-inline" id="myform" action="/submit" method="POST">
            <input class="form-control mr-sm-2" type="text" id="traffic_type"
name="traffic_type" style="display:none;">
          </form>
```

```
<form class="form-inline">
  <div class="dropdown show">
    <a class="btn btn-info dropdown-toggle" href="#" role="button" id="choose_traffic_form" data-toggle="dropdown" aria-haspopup="true" aria-expanded="false" style="width:150px;">
      Select
    </a>
    <div class="dropdown-menu" aria-labelledby="dropdownMenuLink">


      <a class="dropdown-item" onclick="choose_traffic('Probe');">Probe</a>


      <div class="dropdown-divider"></div>
      <a class="dropdown-item" onclick="choose_traffic('Normal');">Normal traffic</a>
    </div>
  </div>
</form>
<li class="nav-item">
  <a class="nav-link" href="#">Traffic features</a>
</li>
</ul>
</nav>
```

```
<!-- Main Form -->
<div class="container" style="margin-top:15px;">
  <div class="card">
      <div class="card-body" style="overflow-y:scroll;">
        <form>
          <div class="form-row">
            <div class="form-group col-md-6">
              <label>Duration</label>
              <input type="number" class="form-control" id="dur" placeholder="Length of
time duration of the connection (0 - 54451)">
            </div>
            <div class="form-group col-md-6">
              <label>Protocol Type</label>
              <select class="form-control" id="prot" required="">
                <option disabled="" selected="">Protocol used in the connection</option>
                <option id="tcp">1. TCP</option>
                <option id="udp">2. UDP</option>
                <option id="icmp">3. ICMP</option>
              </select>
            </div>
          </div>
```

```
<div class="form-row">
  <div class="form-group col-md-6">
    <label>Service</label>
    <input type="text" class="form-control" id="service"
placeholder="Destination network service used">
  </div>
  <div class="form-group col-md-6">
    <label>Flag</label>
    <input type="text" class="form-control" id="flag" placeholder="Flag Type
value type">
  </div>
</div>
<div class="form-row">
  <div class="form-group col-md-6">
    <label>Src Bytes</label>
    <input type="number" class="form-control" id="srcbytes"
placeholder="Number of data bytes transferred from source to dest">
  </div>
  <div class="form-group col-md-6">
    <label>Dstn Bytes</label>
    <input type="number" class="form-control" id="dstnbytes"
placeholder="Number of data bytes transferred from dest to source">
```

```
  </div>
</div>


<div class="form-row">
  <div class="form-group col-md-6">
    <label>Logged In</label>
    <select class="form-control" id="login_status" required="">
      <option disabled="" selected=""><span  style="color:#ccc">Login
Status</span></option>
      <option id="out">0. Logged out</option>
      <option id="in">1. Logged in</option>
    </select>
  </div>
  <div class="form-group col-md-6">
    <label>Wrong Fragment</label>
    <select class="form-control" id="wrong" required="">
      <option disabled="" selected="">Total number of wrong fragments in this
connection</option>
      <option id="wrng0">0</option>
      <option id="wrng1">1</option>
      <option id="wrng2">2</option>
      <option id="wrng3">3</option>
```

```
    </select>
  </div>
</div>


<div class="form-row">
  <div class="form-group col-md-6">
    <label>Same Destn Count</label>
    <input type="number" min=0 max=511 class="form-control" id="count"
placeholder="Number of connections to the same destination">
  </div>
  <div class="form-group col-md-6">
    <label>Same Port Count</label>
    <input type="number"  min=0 max=511 class="form-control" id="srv_count"
placeholder="Number of connections to the same service (port number)">
  </div>
</div>


<div class="form-row">
  <div class="form-group col-md-12">
    <button href="/" class="btn btn-warning" style="margin-left:37%;">Reset
Form</button>
```

```
                <a class="btn btn-success" style="margin-left:30px;color:white;"
onclick="submit_form()">Submit</a>

              </div>

            </div>

          </form>

          <p style="text-align:center;"><b>NOTE:</b> We've shown only selected features
which have more significance, for other features visit: <a href="/features">Feature
Descriptions</a></p>

        </div>

      </div>

    </div>

  </body>

  </html>
<!DOCTYPE html>

<html lang="en">

<style type="text/css">



</style>

<head>

  <title>Result</title>

  <meta charset="utf-8">

  <link rel="shortcut icon" href="./static/images/favicon.png">
```

```
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js"></script>

  <script
src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.16.0/umd/popper.min.js"></script>

  <script src="https://maxcdn.bootstrapcdn.com/bootstrap/4.4.1/js/bootstrap.min.js"></script>

  <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/4.4.1/css/bootstrap.min.css">



  <script type="text/javascript">

    function find_text(){

    var text = document.getElementById("Search").value;

    console.log(text);

    find(text);}

  </script>



  <style>

    .card{

      border:1px solid #0068d0;

      border-radius: 1em;

    }

  </style>



</head>
```

```
<body>


  <nav class="navbar navbar-expand-sm navbar-light fixed-top" style="background-color:
#e3f2fd;">

    <!-- Brand/logo -->

    <ul class="navbar-nav mr-auto">

     <li><a class="navbar-brand" href="/"><img src="./static/images/logo.png" alt="Logo"
style="height:40px;"></a></li>

    </ul>


    <!-- Links -->

    <ul class="navbar-nav mr-right">

      <li class="nav-item dropdown">

        <a class="nav-link" href="/">Stimulate Attack</a>

      </li>


    <form class="form-inline">

      <input class="form-control mr-sm-2" type="text" placeholder="Search" id="Search">

    </form>

    <button class="btn btn-outline-success" onclick="find_text()">Search</button>

    </ul>

  </nav>
```

```
<br><br><br><br>

<h2 style="text-align:center;">Results</h2>

<h5 style="text-align:center;">Expected outcome: <span id="exp"></span></h5>

<br>


<div class="container">

  <div class="card">

      <div class="card-body">


        <p>Normal Traffic</p>

        <div class="progress"  style="margin-top:-10px;margin-bottom:10px;"><div
class="progress-bar progress-bar-striped bg-warning" id="rnd_frst_normal" role="progressbar"
style="width: 75%" aria-valuenow="75" aria-valuemin="0" aria-
valuemax="100">75%</div></div>


        <p>Probe Attack</p>

        <div class="progress" style="margin-top:-10px;margin-bottom:10px;"><div
class="progress-bar progress-bar-striped bg-warning" id="rnd_frst_probe" role="progressbar"
style="width: 75%" aria-valuenow="75" aria-valuemin="0" aria-
valuemax="100">75%</div></div>

      </div>

   </div>
```

```
  </div>
  <br>
  <br>
</body>
</html>


<script>
var js = JSON.parse('{{ dict | tojson | safe}}');
document.getElementById("exp").innerHTML=js["expected"];
var attacks = ["normal","dos","r2l","u2r","probe"];
for(var i = 0; i < 5; i++) {
  document.getElementById("rnd_frst_"+attacks[i]).innerHTML=js[attacks[i]]+"%";
  document.getElementById("rnd_frst_"+attacks[i]).style.width=js[attacks[i]]+"%";
}
var res = js["predictions"];
if(res=="normal"){
  document.getElementById("rnd_frst_"+res).classList.remove('bg-warning');
  document.getElementById("rnd_frst_"+res).classList.add('bg-success');
  document.getElementById("result").innerHTML="Normal Traffic";}
else{
  document.getElementById("rnd_frst_"+res).classList.remove('bg-warning');
```

```
  document.getElementById("rnd_frst_"+res).classList.add('bg-danger');

  document.getElementById("result").innerHTML="Attack detected";}

</script>


function find_text(){

    var text = document.getElementById("Search").value;

    console.log(text);

    find(text);

  }


function choose_traffic(traffic){

    document.getElementById("choose_traffic_form").innerHTML = traffic;

    document.getElementById("traffic_type").value = traffic;

    // ['duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes', 'land', 'wrong_fragment',
'logged_in', 'count', 'srv_count', 'Class']

    var normal = [[0,'udp','domain_u','SF',42,42,0,"out",13,19,'normal']];

    normal.push([0,'icmp','ecr_i','SF',40008,0,0,"in",3,3,'normal']);

    normal.push([0,'udp','private','SF',1,26,0,"in",15,4,'normal']);

    normal.push([0,'tcp','http','SF',209,762,0,"in",17,18,'normal']);

    normal.push([0,'udp','private','SF',54,55,0,"out",509,509,'normal']);

    //
```

```
var dos = [[0,'tcp','name','REJ',0,0,0,"out",203,12,'dos']];
dos.push([7638,'tcp','telnet','SF',0,44,0,"out",1,1,'dos']);
dos.push([0,'tcp','http','S0',0,0,0,"out",82,82,'dos']);
dos.push([0,'tcp','ftp_data','S0',0,0,0,"out",24,4,'dos']);
dos.push([0,'tcp','finger','RSTO',0,0,0,"out",196,9,'dos']);
//
var u2r = [[135,'tcp','telnet','SF',353,5769,0,"in",1,1,'u2r']];
u2r.push([58,'tcp','telnet','SF',2615,4233,0,"in",1,1,'u2r']);
u2r.push([35,'tcp','ftp','SF',96,533,0,"in",1,1,'u2r']);
u2r.push([84,'tcp','telnet','SF',2742,5433,0,"in",1,1,'u2r']);
u2r.push([154,'tcp','telnet','SF',1510,3349,0,"out",1,1,'u2r']);
//
var r2l = [[4,'tcp','pop_3','SF',28,93,0,"in",1,1,'r2l']];
r2l.push([4,'tcp','pop_3','SF',32,93,0,"in",1,1,'r2l']);
r2l.push([288,'tcp','ftp','SF',160,597,0,"out",1,1,'r2l']);
r2l.push([0,'tcp','telnet','SF',124,174,0,"out",2,2,'r2l']);
r2l.push([0,'tcp','pop_3','SF',30,217,0,"in",1,1,'r2l']);
//
var probe = [[0,'tcp','telnet','REJ',0,0,0,"out",1,4,'probe']];
probe.push([0,'tcp','uucp','SH',0,0,0,"out",1,1,'probe']);
probe.push([0,'tcp','other','REJ',0,0,0,"out",480,1,'probe']);
```

```
  probe.push([0,'tcp','sunrpc','REJ',0,0,0,"out",1,3,'probe']);

  probe.push([0,'tcp','other','REJ',0,0,0,"out",505,1,'probe']);


  var json = {"Normal":normal, "DoS":dos, "R2L":r2l, "U2R": u2r, "Probe":probe}

  var random = Math.floor(Math.random() * 10)%5;

  console.log(json);

  console.log(traffic);

  var element = json[traffic][random];


  document.getElementById('dur').value=element[0];

  document.getElementById(element[1]).setAttribute('selected','');

  document.getElementById('service').value=element[2];

  document.getElementById('flag').value=element[3];

  document.getElementById('srcbytes').value=element[4];

  document.getElementById('dstnbytes').value=element[5];

  document.getElementById("wrng"+element[6]).setAttribute('selected','');

  document.getElementById(element[7]).setAttribute('selected','');

  document.getElementById('count').value=element[8];

  document.getElementById('srv_count').value=element[9];

}
```

```
function submit_form(){

  document.getElementById("myform").submit();}

import flask

from flask import Flask, render_template, request, redirect

from eval import main

app = Flask(__name__)


#Basic Web Pages

#-----------------------------------------------------------------------------------------

@app.route("/")

@app.route("/index")

def index():

    return render_template("index.html")


@app.route("/features")

def features():

    return render_template("features.html")


@app.route("/analysis")

def analysis():

    return render_template("analysis.html")
```

```
@app.route("/model")
def model():
    return render_template("model.html")
#-----------------------------------------------------------------------------------------------------


@app.route("/submit", methods=['POST'])
def submit():
    if request.method=="POST":
        type = request.form.get("traffic_type")
        expected = type
        type = type.lower()
        print(type)
        attacks = ["normal","dos","r2l","u2r","probe"]
        if type not in attacks:
            return render_template("index.html")
        pred, prob = main(type)


        dict = {"expected":expected,"predictions":attacks[pred], "normal":prob[0], "dos":prob[1],
"u2r":prob[3], "r2l":prob[2], "probe":prob[4]}
        return render_template("result.html",dict=dict)
```

```
# Commands to run

#-----------------------------------------------------------------------------------------

# export FLASK_APP=server.py

# export FLASK_DEBUG=1

# python -m flask run --host 0.0.0.0 --port 5000

#!/usr/bin/env python

# coding: utf-8


# In[1]:

import numpy as np

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score

plt.style.use('fivethirtyeight')

pd.set_option('display.max_columns', None)

pd.set_option('display.max_rows', None)


# In[2]:

data_train_full = "KDDTrain+.txt"

data_test_full = "KDDTest+.txt"
```

```
# In[3]:

df_train_full = pd.read_csv(data_train_full)

df_test_full = pd.read_csv(data_test_full)

df_train_full.head()


# In[4]:

NSL_KDD_columns = ['duration', 'protocol_type', 'service', 'flag', 'src_bytes',

    'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot',

    'num_failed_logins', 'logged_in', 'num_compromised', 'root_shell',

    'su_attempted', 'num_root', 'num_file_creations', 'num_shells',

    'num_access_files', 'num_outbound_cmds', 'is_host_login',

    'is_guest_login', 'count', 'srv_count', 'serror_rate',

    'srv_serror_rate', 'rerror_rate', 'srv_rerror_rate', 'same_srv_rate',

    'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_count',

    'dst_host_srv_count', 'dst_host_same_srv_rate',

    'dst_host_diff_srv_rate', 'dst_host_same_src_port_rate',

    'dst_host_srv_diff_host_rate', 'dst_host_serror_rate',

    'dst_host_srv_serror_rate', 'dst_host_rerror_rate',

    'dst_host_srv_rerror_rate', 'class', 'classnum']
```

```
# In[5]:

df_train_full.columns = NSL_KDD_columns

df_test_full.columns = NSL_KDD_columns


# In[6]:

from scipy.stats import zscore

df = zscore(df_train_full['classnum'])


# In[7]:

print("Z-score Calculation based on class to check outliers")

df


# In[8]:

df_train_full.info()


# In[9]:

df_test_full.info()


# In[10]:

df_train_full.nunique()
```

```
# In[11]:

df_test_full.nunique()


# In[12]:

df_train_full.isnull().sum()


# In[13]:

df_test_full.isnull().sum()


# In[14]:

NSL_KDD_columns_categorical = ['protocol_type', 'service', 'flag', 'class']


NSL_KDD_columns_int = ['duration',  'src_bytes',

      'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot',

      'num_failed_logins', 'logged_in', 'num_compromised', 'root_shell',

      'su_attempted', 'num_root', 'num_file_creations', 'num_shells',

      'num_access_files', 'num_outbound_cmds', 'is_host_login',

      'is_guest_login', 'count', 'srv_count',  'dst_host_count',

      'dst_host_srv_count',   'classnum']


NSL_KDD_columns_float = ['serror_rate', 'srv_serror_rate', 'rerror_rate', 'srv_rerror_rate',
```

```
    'same_srv_rate', 'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_same_srv_rate',
    'dst_host_diff_srv_rate', 'dst_host_same_src_port_rate',
    'dst_host_srv_diff_host_rate', 'dst_host_serror_rate',
    'dst_host_srv_serror_rate', 'dst_host_rerror_rate',
    'dst_host_srv_rerror_rate']


# In[15]:
from sklearn.feature_selection import SelectKBest
temp = df_train_full['protocol_type'].value_counts()
a = temp.index
b = temp.values
zipped = list(zip(a, b))
d = pd.DataFrame(zipped, columns=['protocol_type', 'occurances'])
sns.barplot(data=d, x='protocol_type', y='occurances')


# In[16]:
temp = df_train_full['service'].value_counts()
a = temp.index
b = temp.values
zipped = list(zip(a, b))
d = pd.DataFrame(zipped, columns=['service', 'occurances'])
```

```
fig, ax = plt.subplots(figsize=(15, 6))

sns.barplot(data=d, x='service', y='occurances')

fig.tight_layout()

plt.xticks(rotation=90)

plt.show()

#ax.set_xlabel()

#ax.set_ylabel()


# In[17]:

temp = df_train_full['flag'].value_counts()

a = temp.index

b = temp.values

zipped = list(zip(a, b))

d = pd.DataFrame(zipped, columns=['flag', 'occurances'])

fig, ax = plt.subplots()

sns.barplot(data=d, x='flag', y='occurances')

fig.tight_layout()

plt.xticks(rotation=90)

plt.show()
```

```
# In[18]:

# categorical feature 'class'

temp = df_train_full['class'].value_counts()

a = temp.index

b = temp.values

zipped = list(zip(a, b))

d = pd.DataFrame(zipped, columns=['class', 'occurances'])

fig, ax = plt.subplots()

sns.barplot(data=d, x='class', y='occurances')

fig.tight_layout()

plt.xticks(rotation=90)

plt.show()


# In[19]:

print(df_train_full[NSL_KDD_columns_int].nunique())


# In[20]:

list1 = ['land', 'wrong_fragment', 'urgent', 'num_failed_logins',

         'logged_in', 'root_shell', 'su_attempted', 'num_shells',

         'num_outbound_cmds', 'is_host_login', 'is_guest_login']
```

```
list2 = ['duration', 'src_bytes','dst_bytes', 'hot',
         'num_compromised', 'num_root', 'num_file_creations',
         'num_access_files', 'count', 'srv_count', 'dst_host_count',
         'dst_host_srv_count', 'classnum']


# In[21]:
for i in list1:
    print('feature name:',i,'\n', df_train_full[i].value_counts(ascending=False), '\n\n\n')


# In[22]:
DoS_attacks =
['apache2','back','land','neptune','mailbomb','pod','processtable','smurf','teardrop','udpstorm','worm
']
Probe_attacks = ['ipsweep','mscan','nmap','portsweep','saint','satan']
U2R = ['buffer_overflow','loadmodule','perl','ps','rootkit','sqlattack','xterm']
R2L =
['ftp_write','guess_passwd','http_tunnel','imap','multihop','named','phf','sendmail','snmpgetattack','
snmpguess','spy','warezclient','warezmaster','xclock','xsnoop']
```

```
# In[23]:

df_train_full['class'].replace(['apache2','back','land','neptune','mailbomb','pod',

                                'processtable','smurf','teardrop','udpstorm','worm'],

                                'DoS_attack', inplace=True)

df_train_full['class'].replace(['ipsweep','mscan','nmap','portsweep','saint','satan'],

                                'Probe_attacks', inplace=True)

df_train_full['class'].replace(['buffer_overflow','loadmodule','perl','ps','rootkit',

                                'sqlattack','xterm'], 'U2R', inplace=True)

df_train_full['class'].replace(['ftp_write','guess_passwd','http_tunnel','imap',

                                'multihop','named','phf','sendmail','snmpgetattack',

                                'snmpguess','spy','warezclient','warezmaster','xclock',

                                'xsnoop'], 'R2L', inplace=True)

df_train_full['class'].replace('normal', 'Normal', inplace=True)

# In[ ]:


# In[24]:

df_train_full.drop(df_train_full[(df_train_full['class']=='U2R')].index,inplace=True)

df_train_full.drop(df_train_full[(df_train_full['class']=='DoS_attack')].index,inplace=True)

df_train_full.drop(df_train_full[(df_train_full['class']=='R2L')].index,inplace=True)
```

```
df_test_full.drop(df_train_full[(df_train_full['class']=='U2R')].index,inplace=True)

df_test_full.drop(df_train_full[(df_train_full['class']=='DoS_attack')].index,inplace=True)

df_test_full.drop(df_train_full[(df_train_full['class']=='R2L')].index,inplace=True)

df_train_full[(df_train_full['class']=='Normal')].head()


# In[25]:

df_train_full['class'].value_counts()

df_train_full.rename(columns={'class':'Label'},inplace=True)

df_test_full.rename(columns={'class':'Label'},inplace=True)


# In[26]:

Trained_attack = df_train_full.Label.map(lambda a: 0 if a == 'Normal' else 1)

Test_attack=df_test_full.Label.map(lambda a: 0 if a == 'Normal' else 1)


df_train_full['Label'] = Trained_attack

df_test_full['Label'] = Test_attack


# In[27]:

df_train_full['Label']
```

```
# In[28]:

df_train_full.plot(kind='box',subplots=1,layout=(8,5),figsize=(20,40))

plt.show()


# In[29]:

df_train_full.select_dtypes(['object']).columns


# In[30]:

from sklearn import preprocessing

le=preprocessing.LabelEncoder()

clm=['protocol_type', 'service', 'flag', 'Label']

for x in clm:

    df_train_full[x]=le.fit_transform(df_train_full[x])

    df_test_full[x]=le.fit_transform(df_test_full[x])


# In[31]:

from sklearn.metrics import accuracy_score,f1_score


x_train=df_train_full.drop('Label',axis=1)

y_train=df_train_full["Label"]
```

```
x_test=df_test_full.drop('Label',axis=1)

y_test=df_test_full["Label"]


# In[32]:

from sklearn.feature_selection import mutual_info_classif


mutual_info = mutual_info_classif(x_train, y_train)

mutual_info = pd.Series(mutual_info)

mutual_info.index = x_train.columns

mutual_info.sort_values(ascending=False)


# In[33]:

def select():

    best_one=[]

    if(len(val_list)>1):

        best_values=sorted_value(val_list/2)

        for i in best_values:

            best_one.append(pop_lst(value_list[i]))

        return best_one


sel_five_cols = SelectKBest(mutual_info_classif, k='all')
```

```
sel_five_cols.fit(x_train, y_train)

print("The selected features Are:")


# In[34]:

x_train.columns[sel_five_cols.get_support()]


# In[35]:

col=['service', 'flag', 'src_bytes', 'dst_bytes', 'logged_in',

     'same_srv_rate', 'diff_srv_rate', 'dst_host_srv_count',

     'dst_host_same_srv_rate', 'dst_host_diff_srv_rate']

x_train=x_train[col]

x_test=x_test[col]


# In[36]:

from sklearn.preprocessing import MinMaxScaler

from lib.utils import *

scaler = MinMaxScaler()

X_train= scaler.fit_transform(x_train)

X_test= scaler.fit_transform(x_test)
```

```
# In[ ]:


# In[37]:

ac=[]

f=[]


from sklearn.naive_bayes import GaussianNB

model=GaussianNB()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

ac.append(accuracy_score(y_pred,y_test)*100)


# In[ ]:


# In[39]:

from keras import layers

from keras.models import Sequential


model_ = Sequential()

model_.add(layers.Dense(20, input_dim=10, activation='relu'))

model_.add(layers.Dense(10,  activation='tanh'))
```

```
model_.add(layers.Dense(1024, activation='relu'))

model_.add(layers.BatchNormalization())

model_.add(layers.Dropout(0.5))

model_.add(layers.Dense(1, activation='sigmoid'))

model_.compile(loss='binary_crossentropy',

               optimizer='adam',

               metrics=['accuracy'])

model_.summary()

classifier_nn = model_.fit(X_train,y_train,

                    epochs=10,

                    verbose=True,

                    validation_data=(X_test, y_test),

                    batch_size=15)

ac.append(accuracy_score(y_pred,y_test,sample_weight=0.2)*100)


# In[40]:

ac

# In[41]:

x=['Naive Bayes','CNN']

import matplotlib as plt

plt.style.use('ggplot')
```

```
plt.rcParams["figure.figsize"] = (10,7)

y=ac

import numpy as np

import seaborn as sns

ax=sns.barplot(x,y)

ax.set_title('Accuracy comparison')

ax.set_ylabel('Accuracy')

low = min(y)

high = max(y)


# In[42]:

pred=model.predict(X_test)


# In[44]:

for i in range(1,20):

    if(pred[i]==0):

        print("No attack detected")

    else:

        print("Probe attack detected")


# In[ ]:
```